



Review

A theoretical review on solving algebra problems

Xinguo Yu ^{ID}*, Weina Cheng ^{ID}, Chuanzhi Yang ^{ID}, Ting Zhang ^{ID}

Central China Normal University, 152 Luoyu Road, Hongshan District, Wuhan, 430079, Hubei, China

ARTICLE INFO

Keywords:

Problem solving
State transform theory
State transform graph
State graph
Perspective confusion comparison

ABSTRACT

Solving algebra problems (APs) continues to attract significant research interest as evidenced by the large number of algorithms and theories proposed over the past decade. Despite these important research contributions, however, the body of work remains incomplete in terms of theoretical justification and scope. The current contribution intends to fill the gap by developing a review framework that aims to lay a theoretical base, create an evaluation scheme, and extend the scope of the investigation. This paper first develops the State Transform Theory (STT), which emphasizes that the problem-solving algorithms are structured according to states and transforms unlike the understanding that underlies traditional surveys which merely emphasize the progress of transforms. The STT, thus, lays the theoretical basis for a new framework for reviewing algorithms. This new construct accommodates the relation-centric algorithms for solving both word and diagrammatic algebra problems. The latter not only highlights the necessity of introducing new states but also allows revelation of contributions of individual algorithms - obscured in prior reviews without this approach.

A review of AP solving algorithms (2014 to date) is subsequently enhanced by applying the STT specifically designed to analyze individual and collective algorithms for states and transforms. Furthermore, the State Transform Analysis (STA) is a core function in the identification of progress in terms of states and transforms. Thirdly, the Perspective Confusion Comparison (PCC) is developed to extend the application of STA to add capabilities for systematic and individual evaluation of transforms, algorithms, and approaches. This is a new evaluation method of being different from other methods that can do more in-depth evaluation for decomposed algorithms for solving problems with multiple types of inputs. Finally, this work identifies several research directions by extracting benefit from theoretical reviews. This work significantly contributes to the advancement of AP-solving by providing the mechanism for identifying and understanding the key contributors to building high-performance problem-solving algorithms at the three levels of transform, algorithm, and approach.

1. Introduction

As interest in solving algebra problems (APs) is continuing to gain momentum, the absence of theoretical justifications for new works is more keenly felt than ever. It is thus imperative and timely that a theoretical perspective is applied to the AP-solving algorithms because the relevant conditions are present. These include the fact that significant numbers of algorithms for solving APs have appeared in the literature over the past six decades as well as the existence of several important review papers that provided substantial background knowledge for this review (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a). For example, several papers developed the methods and algorithms for solving text-diagram problems (He et al., 2020; Jian et al., 2019; Yu et al., 2022), which extended the scope of problems from Algebra

Word Problems into Algebra Problems (mainly from mathematics, physics, and chemistry in basic education).

The papers referred to these overviews, however, demonstrate that current research is still unsatisfactory in terms of inclusions, time span, and theoretical underpinnings despite attempts justifying AP-solving algorithms at model-level of which the state action frameworks developed by Xia and Yu (2021), Yu et al. (2023) are significant examples. As many others have done, the authors stress the importance of contributions made by action techniques but pay scant attention to “states”, which, in fact, play an essential role in determining the properties of solving algorithms, providing further justification for our work.

The development of a theory-based review of algorithms for solving APs is, thus, justified by the research communities’ need, significant number of models and paradigms that have appeared, creating the relevant conditions for the development of a theory: (a) a large body of

* Corresponding author at: National Engineering Research Center for E-learning, Central China Normal University, Wuhan, 430079, China.

E-mail addresses: xgyu@ccnu.edu.cn (X. Yu), wcheng@mails.ccnu.edu.cn (W. Cheng), canola@mails.ccnu.edu.cn (C. Yang), ting.zhang@ccnu.edu.cn (T. Zhang).

<https://doi.org/10.1016/j.eswa.2025.128789>

Received 29 January 2024; Received in revised form 18 September 2024; Accepted 24 June 2025

Available online 6 July 2025

0957-4174/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

algorithms exists, (b) a range of review papers have laid the groundwork (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a), and (c) the work of Xia and Yu (2021), Yu et al. (2023) has provided specific paradigms that have the capacity to be extended to a theory.

As none of the review papers theoretically justified their work, this creates an urgent need for a theory that offers a basis of the analysis of AP-solving algorithms, explains their processes, provides a uniform basis for comparison, and provides theoretical guidance for the further development of AP-solving algorithms. To achieve this, we draw on the work on state transition paradigms in Xia and Yu (2021), Yu et al. (2023) which provides a starting point for the development of a theory for reviewing AP-solving algorithms.

In the first step, to construct the State Transform Theory, this paper draws from the state action paradigm proposed in Yu et al. (2023). This construct considers problem-solving in three phases, with the first and most significant consisting of states and actions. This first phase provides a logical basis for progression to the subsequent phases of problem-solving represented by “transforms” from the state to the algorithmic output. We posit that the presence of a “transform” in the work structure of an algorithm indicates its consistency with a theoretical model.

Some researchers have tried to use the state transition paradigm to explain algorithms in abstract level, through a mathematical model that describes the dynamic process of a finite state machine (Gill, 1960). The state transition paradigm is used to reveal the structure and properties of algorithms (Luan & Li, 2001), to design the algorithm (Yu et al., 2023; Zhou et al., 2020), or to analyze algorithms (Luan & Li, 2001). The state transition paradigm (borrowed from finite state machine) is used as a theoretical model to describe some constructs of algorithms. Both its states and transitions are the abstract concepts whereas states and transitions in “finite state machine” are concrete physical or mathematical contents. States express the abstract common properties of the intermediate outcomes of algorithms; transitions express the theoretical methods of fulfilling state transition. To rephrase, for algorithm analysis the state transition paradigm is a mental model for abstract phenomena but not a concrete physical configuration.

Hence, we propose the State Transform Theory (STT), which uses “state” and “transform” as key constructs to build State Transform Graphs (STGs). These graphs visualize the structure of an algorithm and confirms its decomposability. Building on the STT, we introduce the State Transform Analysis method, a graphical approach to analyze algorithms in three steps. First, we list all representative algorithms since 2014, identifying their respective states and key transforms. Second, we represent each algorithm with its individual STG, followed by an aggregated graph visualizing all collected algorithms. This aggregated graph is abstracted to create a state graph. Additionally, the four approaches are uniformly defined on the aggregated graph, and a taxonomy tree of the algorithms is generated. Both the aggregated graph and the state graph enable us to draw global conclusions about the algorithms. To further understand the solving algorithms, we examine the progression of states and transforms for each of the four approaches. For comparative analysis of perspective confusion, we classify algebra problems into different types, allowing us to evaluate the distribution of transforms, algorithms, and approaches across these types. This provides a performance landscape for solving algebraic problems (APs). However, this paper presents only sample evaluations rather than a complete analysis, due to the focus of this study.

1.1. Contributions

This paper integrates and develops several existing paradigms into a State Transform Theory by generalizing the paradigms to all representative algorithms and making statements about properties of individual algorithms and collective algorithms. Using this theory as a basis, a re-

view of research into solving algebra problems is conducted whereby algorithms are reviewed at three level: transform, algorithm, and approach. Through conducting this novel review, this paper makes four contributions as follows:

- It develops the State Transform Theory (STT), serving as the theoretical basis for reviewing AP-solving algorithms.
- It proposes the State Transform Analysis method to depict and analyze both individual and collective algorithms based on the STT.
- It proposes the review paradigm to define and review approaches that analyze the progression of approaches by reviewing the evolution to their states and transforms.
- It proposes the Perspective Confusion Comparison to critically compare transforms, algorithms, and approaches, a method superior to the ablation experiment.

1.2. Organization

The organization of the remainder of the paper is as follows. The related work is reviewed in Section 2. The STT is built in Section 3 while Section 4 presents the State Transform Analysis method of reviewing algorithms in which State Transform Graphs are used to depict individual and collective algorithms based on the STT. Section 5 reviews the approaches of AP-solving algorithms through narrating the evolution of states and transforms. Section 6 presents the Perspective Confusion Comparison on transforms, algorithms, and approaches. Section 7 concludes the paper and discusses future research directions.

2. Related work

Solving APs is longstanding research problem which continues to be a challenge. Despite dating back to the 1960s, the interest in solving APs has intensified only recently with the advance of Artificial Intelligence (AI) and the demand for more intelligent education. During this long research history, seven survey papers have reviewed problem-solving algorithms with the aim of identifying the difficulties, as well as mining and generally useful techniques to build a body of knowledge (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a) which will be reviewed in this section. Different from the survey papers, several researchers-initiated work of developing a model to explain their algorithms but not historical algorithms (Xia & Yu, 2021; Yu et al., 2023). These papers inspire this work to build a theory to not only explain individual algorithms but also analyze collective algorithms, further guide algorithm development and algorithm evaluation.

In computer science, several algorithm evaluation paradigms have previously been proposed to evaluate types of algorithms. Generally, the researchers adopted a benchmark paradigm to evaluate AI algorithms (Campbell, 1995; Mendling et al., 2021). As these attempts that were based on ‘post’ and competitive evaluation and comparison, they were unable to reveal the inner structure and techniques of algorithms, nor could they identify the theoretical relations among them. This research gap inspires us to develop a perspective confusion comparison in this paper.

2.1. Accumulation of review knowledge for AP solving

A significant body of knowledge has been accumulated over the past six decades on AP-solving found in seven review papers (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a) each of which has contributed today’s understanding of these issues within the research community.

Carifio and Nasser (1994) published the first review paper on solving APs wherein they proposed a two-stage review scheme, which divides

the process of solving APs into representation and solution stages. They placed emphasis on identifying direct and inferred problem representation to obtain answers at the theoretical level

Almost fifteen years after (Carifio & Nasser, 1994; Mukherjee & Garain, 2008) took the issue up again with their review. Again, the authors followed a two-stage review scheme although this time analyzing the workflow. This paper offered the observation that understanding natural language is the core step to solving mathematical problems (including algebra, geometry, physics, mechanics, etc.), consequently reviewing methods employed for understanding descriptions of problems. The authors also included an interesting discussion of the progress of rule-based methods published before 2008.

This attempt at categorization was advanced by Mandal and Naskar (2019) who proposed two schemes for reviewing literature on AP-solving of which the first one was a stage scheme that divided the history of the research on these specific problem-solving algorithms into two periods: Early Stage (1964–2006) and New Era (2007–2017). They concluded that in the Early Stage, researchers viewed solving APs either from a point of cognitive science, psychology, or education. Researchers developed multiple systems and algorithms for solving limited numbers of specific types of simple problems aimed at simulation of the processes by which the human students solve problems. In consequence, the proposed methods are mainly rule-based targeting APs with specific processes. The algorithms developed in New Era were mainly published between 2007 and 2017 (Mandal & Naskar, 2019).

During this period, machine learning based methods came into being and were applied to the development of problem-solving algorithms. This scheme classifies the algorithms into the following four categories based on the characteristics of the technique used to build them: 1) equation template-based; 2) equation tree-based; 3) entity and state transition-based and 4) tag/meaning based. The gained insights from this work are of significance for this paper as the first survey identified that solvers have two phases of problem understanding and symbolic solver (symbolic solver originally called solution stage). These three reviews agreed that algorithms are largely defined by their construction techniques, and these have a strong timestamp. They also agreed that all solving algorithms have two phases: problem understanding and symbolic solving. Whilst some of these insights are not precisely new, they re-enforce our current understanding and are, therefore, useful for our work.

In the same year as Mandal and Naskar (2019), Kotwal and Patnaik (2019) used a quantity scheme to classify problems into categories according to the number of unknowns in each problem, subsequently applying a stage scheme to divide the algorithms into first generation and data driven. This research generated the idea to divide the AP corpus into groups according to their difficulty based on the belief that the number of unknowns in each problem is an indicator of the difficulty. This problem division inspires this paper to divide APs into more categories according to more indicators.

A year later, Zhang et al. (2020a) re-adopted a cognitive approach by developing a systematic “semantic gap” review. The authors think that the difficulty in solving APs lies in the semantic gap that exists between human readable words and machine understandable logic. They, again, used a stage scheme, this time to classify the algorithms into, three generations according to the techniques underpinning them: the first generation (1960–2010) was characterized by rule-based matching, the second (2011–2017) by semantic parsing, feature engineering and statistical learning, and the third generation (2017–2019) by Deep Learning (DL) and Reinforcement Learning (RL). This paper also used the unknown quantity scheme to divide the solvers into two categories, arithmetic word solvers (AWS) capable of solving problems with one unknown, and equation set solvers (more than one unknown). The work further provided a confusion performance comparison of two different types of algorithms on different datasets and a discussion of advantages and disadvantages. The researchers used, in addition, a confusion comparison to discover the performances that solvers work on the various

datasets. The take-away from this paper was that chronology schemes have repeatedly been used in surveys, indicating that.

- the solvers have a strong timestamp.
- the scheme aids classification of stages and categorization of AP-solving.
- the confusion comparison is a useful way to identify the actions that solvers carry out on datasets.

Also reviewing based on methodology were (Faldut et al., 2021) who revisited APs, identifying the use of neural networks as an indicator of a quality solver. The paper further inspected non-neural and neural methods narrated in natural language applied as categorizer (1) direct answer generator, (2) expression tree generator for inferring answers, and (3) template retriever for answer computation. This work showed that neural methods can output two types of results: direct answer and expression tree. This indicates that some neural networks can transform the vector sequence into a text-based math expression or expression tree with one unknown.

Lu et al. (2023) conducted a survey on mathematical reasoning tasks and neural networks for mathematical reasoning which identified the four mathematical reasoning tasks: math word problem solving, theorem proving, geometry problem solving, and math question answering. It identified four types of neural networks for mathematical reasoning: seq2seq-based neural networks, graph-based networks, attention-based networks, and other neural networks.

Clearly, these works have made significant contributions in one or more areas; however, they would have benefited from

- a theoretical justification of their review schemes, absence of which prevents justification of fairness, completeness, and use of the work for guidance.
- a more comprehensive review which could have included relation-centric algorithms and algorithms for solving APs narrated through text and diagrams.
- identification and exploration of intermediate representations of the problems.
- a technique-based scaffold would have been of benefit.
- a perspective-based understanding of AP-solving areas, superior to the single-level confusion comparison.

Motivated by need, this paper conducts a theoretical review, based on the state transform theory (STT), which includes the identification of the intermediate representations, multiple level of techniques, and perspective confusion comparison for representative algorithms designed to solve APs.

2.2. Theory development in AP solving

Creating general model to analyze algorithms is a fascinating research problems due to the algorithm pseudo codes and structures are not built on the sound theory or uniform terms. Based on this cognition, Luan and Li (2001) proposed a formal specification based on first-order language to describe the internal state transition of algorithms. Although promising, this work remains in its early stage without further development or application. This state transition is a certain similarity with finite state machine. A finite state machine is a computational model that can be implemented with hardware or software and used to simulate logic structure and certain computer programs (Gill, 1960). Finite state machines can model problems in many fields including mathematics, artificial intelligence, games, and linguistics. There is an understanding that representations and techniques appearing in algorithms have a certain similarity with states and transitions in finite state machines (Gill, 1960). Based on this cognition, some researchers used state transition model to build some algorithms since they realized that it may be too ambitious to create a state transition model for analyzing general algorithms. The following is one of examples. Zhou et al. (2020) proposed a

state transition algorithm, which was proposed on the concepts of state, state transition and state space representation in modern control theory. Because of its excellent global search ability and fast convergence, the state transition algorithm has since been applied in various optimization problems, which inspires us a new way to improve solving algorithms in the future.

The rich facts in the literature in problem solving shed the light of that there is a model that can uniformly analyze most and main solving algorithms. Especially, All seven survey papers discussed above provided the substantial facts. They all focused on techniques that transform representations, indicating that reviewers uniformly agreed that representations and techniques which transform representations are the main elements for constructing these algorithms, although implied rather than explicitly stated (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a). Several primary research papers provided more facts as follows.

Yu et al. (2019) first proposed an equivalent representation theory, which pointed out that math problem solving is to transform states from problem input to output. During this transition process, several intermediate states will be generated. In the sense of math problem solving, these states are equivalent, that is, theoretically, the solution can be obtained from the input state through operations such as reasoning and calculation. The equivalent representation theory signifies that solving a problem is to transform one representation into its next, representing the seminal idea of algorithm structure of states and transforms.

Xia and Yu (2021) developed an equivalent representation theory developed in Yu et al. (2019) into a state action framework to describe the process of solving APs by algorithms which was built around the development of solvers that solve APs similar to students. For a solving algorithm, its input is the problem described by text and diagram while its output ideally is the solution to the given problem. The solving process aims to change one state to its next state through actions. The framework reveals the highly abstract nature of the cognitive model to explain the process of solving APs represented as text and diagram offering deep insights into the level of abstraction of algorithms. This makes it clear that approaches are defined by determining instances of states, thus providing a formal way to define these.

Yu et al. (2023) further developed the state action framework into a three-phase scheme to design algorithms from abstraction to concreteness. The first phase is to design a solving paradigm that proposes a state action paradigm for solving APs as the solving paradigm. The second phase is to determine the content of each state of the state action paradigm to produce a relation-centric approach to solving AWP. The third phase is to determinate the actions of the relation-centric approach to instantiate an algorithm for solving AWP. Thus, this three-phase scheme works on the understanding that a solving algorithm can be abstracted into elements of states and actions. This represented a preliminary exploration of the work in this paper, and we intend to further develop and integrate the existing schemes into the STT.

The above-mentioned papers have contributed concepts, innovative ideas, and frameworks to building a theory capable of accounting for solving algorithms. By further developing these, this work builds an STT to explain the algorithms for solving APs, reveal the inner main states and transforms, form a new evaluation paradigm, and predict the trend in algorithm development.

2.3. Algorithm evaluation paradigms

Algorithm evaluation is an important topic because it is the basis for choosing the appropriate algorithms to solve the problems facing scientists; further, the algorithm is one of the key pillars in computer science. The first algorithm evaluation paradigm is to evaluate the algorithm through complexity analysis, being the important content in textbooks on algorithm analysis. A look at complexity analysis reveals that it is not applicable to most of the AI algorithms because their inputs

are restricted to a narrow range of sizes, whereas complexity analysis requires that the input size ranges from finite to infinity. In consequence, a second algorithm evaluation paradigm was developed (Mendling et al., 2021), which evaluates algorithms by their performance on benchmark datasets. The benchmark evaluation tests against performance indexes such as accuracy, recall, and running time on benchmark datasets. Such evaluations are seen as competitive tests inviting significant criticism. One of these is that the overemphasis on “state of the art” performance was “allowing researchers to publish dull papers that proposed small variations of existing algorithms and reported their small-but-significant incremental performance improvements in comparison studies” (Mendling et al., 2021). The second criticism is that it provides limited understanding of the reasons for the performance of algorithms. Therefore, in recent years, the ablation experiment has become a requirement for benchmark evaluation paradigms. This is an advance in algorithm evaluation since it offers some transparency in terms of the techniques that have contributed to the algorithm’s performance. However, it is still an “after algorithm” or “external algorithm” comparison that does not consider the constituents (i.e., technique) of algorithms. This has prompted the development of other analysis schemes although they have not yet been fully developed. For example, a formal specification based on the first-order language was proposed to describe the internal state transition of algorithms (Luan & Li, 2001) which has so far not been adopted, to the best of our knowledge. Thus, the limitations of traditional review schemes are stimulating the search for new paradigms for reviewing algorithms. Xia and Yu (2021), for example, have developed an approach review scheme for algorithms that solve arithmetic word problems. This development demonstrated that the instances that instantiate abstract frameworks can be used to uniformly define the approaches instead of using heuristic indicators without technique scaffold. Based on the understanding gained from a thorough evaluation of existing review paradigms and the recognition that none of these would provide support for the development of theoretical work, this paper proposes the development of a new review paradigm capable of reviewing algorithms through identification of states and transforms, graphic analysis, and perspective confusion comparison.

3. State transform theory

The review of the related work in the preceding section discloses that the traditional review methods lack theory foundation. To fill this gap, this section develops the State Transform Theory (STT) as the theory base of the review method proposed in this paper.

3.1. Motivation and justification

In AP-solving, review papers concentrated on the new transforms (i.e., techniques), almost neglected the evaluation of state creation and evolution (Carifio & Nasser, 1994; Faldu et al., 2021; Kotwal & Patnaik, 2019; Lu et al., 2023; Mandal, 2019; Mukherjee & Garain, 2008; Zhang et al., 2020a). This motivates to do a different review with emphasis on the fact that it is capable of creation of state as well as technique.

Some other works with theoretical notions also attracted our attention such as those that developed basic concepts, abstract frameworks, theoretical explanations, theoretical processes for different purposes (Luan & Li, 2001; Xia & Yu, 2021; Yu et al., 2023). Some researchers proposed an equivalent representation theory, which pointed out that mathematical problem solving is a state transition from problem input to output through intermediate states (Yu et al., 2019). This theory shows that several equivalent intermediate states will be generated during the entire transition process of problem solving. Then, the researchers further proposed a state action framework to explain the process of solving APs (Xia & Yu, 2021). This framework consists of states and actions and it defines the approaches of solving algorithms by states. Recently Yu et al. (2023) created another usage of the state action framework. Namely, it proposed a three-phase scheme to develop algorithms for

solving AWPss, which is a scheme to design algorithms from abstract to concreteness. In another view, this three-phase scheme reveals the relationship among framework, approach, algorithm, state, and action. These achievements motivate us to build a theory as the basis of review.

This section builds the State Transform Theory by further refining the theoretical concepts introduced in the previously discussed papers. The theory posits that the structural makeup of states dictates their core properties, and both these structures and the techniques applied to them influence performance. The theory will introduce several new assertions to highlight the essential characteristics of individual algorithms as well as groups of algorithms. First, all representative algorithms developed since 2014 can be modeled as graphs of states and transforms. Second, all such algorithms are decomposable. Third, these algorithms share a common set of states and transforms.

Leveraging the above stated theory accumulation, this paper develops a theoretical framework to review all representative algorithms, which reveals that the states represent the key progresses whereas previous reviews used only transforms (techniques) to depict the progresses in AP solving. Based on this cognition, this paper mines all main states and then all transforms. This theoretical framework further uses crucial types of states to define the approaches of algorithms whereas existing review papers used the transforms (techniques) to define approaches. Our theoretical framework also creates a perspective confusion comparison which surpasses the ablation experiment in two aspects. First, it can reveal which transforms contribute to the performance of the whole algorithm. Second, it provides a perspective confusion comparison of transform, algorithm, and approach, deeper than step and algorithm comparison in the ablation experiment.

3.2. Constructs of state transform theory

The following concepts are the constructs of building state transform theory, in which state and transform are two pillar constructs.

Definition 1 (Corpus). A set of algebra problems (APs), prepared by the research community, is called as a corpus of APs, denoted as $\mathcal{C}$.

In this paper, Algebra Problems are Algebra Word Problems and Algebra Text-Diagram Problems, which are mainly from mathematics, physics, and chemistry in basic education.

Definition 2 (Solving Algorithm). Let $\mathcal{C}$ be a corpus of algebra problems. $\mathcal{A}$ is said to be a solving algorithm of $\mathcal{C}$ if $\mathcal{A}$ can correctly solve a significant percent of elements in $\mathcal{C}$.

Definition 3 (State). A state refers to an intermediate milestone representation generated by a solving algorithm while solving algebra problems, typically denoted a state as $\mathcal{S}$ with subscription.

Definition 4 (Transform). Let $\mathcal{S}_s$ and $\mathcal{S}_e$ be two states and t be a method or scheme that can be instantiated into a process of some solving algorithms that can transform instances in $\mathcal{S}_s$ into instances in $\mathcal{S}_e$ always for a corpus of problems.

“State” and “Transform” are two pillar concepts of the theory. “State” is the general name of various intermediate outcomes produced by solving algorithms and “transform” is an umbrella term of a range of processes. The concrete states and transforms are hidden in the algorithms in the literature. To identify them and to dissemble them from algorithms are two tasks in the next section.

States are the representations of the results of the milestone intermediate goals. The understood form of problems is a sample intermediate goal. The results of such a goal could be a math expression, a set of linear equations, or a set of relations. The vector form of problems becomes another sample intermediate goal if machine learning is used to solve algebra problems.

Here, two example states are on the understood form. The first example is Linear Equations, initially proposed by Kushman et al. (2014).

Linear equations are widely recognized as a representation of the intermediate results when solving arithmetic word problems (Cao et al., 2021; Kushman et al., 2014; Wang et al., 2017; Zhou et al., 2015). So far the vector sequence is the only state for the vector form. The transforms are the methods that should produce the instances of states for algebra problems. The second example is Relation Set, as proposed by Yu et al. (2019). This state is commonly used because it serves as an intermediate result for a broader range of math problems (Gan et al., 2019a; He et al., 2020; Yu et al., 2023, 2019).

3.3. State transform graphs of algorithms

The state transform graph is defined to depict the concern algorithms.

Definition 5 (State Transform Graph). A directed graph, $\mathcal{G} = (\mathcal{S}, \mathcal{T}, \phi)$, is called as a state transform graph, where $\mathcal{S}$ is a set of state nodes and $\mathcal{T}$ is a set of transforms that directly link states, which appear in algorithms for solving algebra problems, $\phi : \{\mathcal{T} \rightarrow (x, y) \mid (x, y) \in \mathcal{S} \times \mathcal{S}, x \neq y\}$ is an incidence function mapping every link to an ordered pair of nodes.

Definition 6 (Decomposable Algorithm). An algorithm is called as a decomposable algorithm if its state transform graph has more than two states.

With these concepts, we have the following three statements.

Statement 1. All the algorithms for solving algebra problems can be depicted by state transform graphs.

Statement 2. All the representative algorithms for solving algebra problems since 2014 in the literature are decomposable.

Statement 3. All the representative algorithms for solving algebra problems since 2014 in the literature share a set of states and transforms.

The lists of algorithms, states, and transforms will be presented in Section 4.2. Three statements will be verified in Section 3.3. Statement 1 and 2 are the core conclusions of the state transform theory because they show that this paper can use the state transform graph to analyze and compare the solving algorithms. Statement 3 further tells us that the task of developing a solving algorithm can be broken into the state design, transform design, and state transform graph design. Thus, we can evaluate states, transforms, algorithms, and approaches in any form of combination.

4. State transform analysis

With the built STT, this section presents a novel State Transform Analysis (STA) applied to algebra problem-solving. It begins with an overview of STA, followed by the real steps of the analysis.

4.1. Overview of STA

This section provides an explanation of State Transform Analysis (STA). To begin, the STA is defined as follows:

Definition 7 (State Transform Analysis (STA)). The State Transform Analysis is a method for analyzing algorithms based on State Transform Theory. It involves examining individual algorithms to generate state transform graphs that represent the transitions between different states during execution. These graphs are then aggregated to identify common properties and patterns across multiple algorithms.

The STA performs the following tasks: First, it identifies 47 representative solving algorithms from the literature since 2014. Next, it identifies 15 non-default states and 42 transforms. Following that, it constructs state transform graphs for all 47 algorithms and combines them into a single, aggregated state transform graph. Furthermore, it abstracts this aggregated graph into a state graph and defines four distinct approaches.

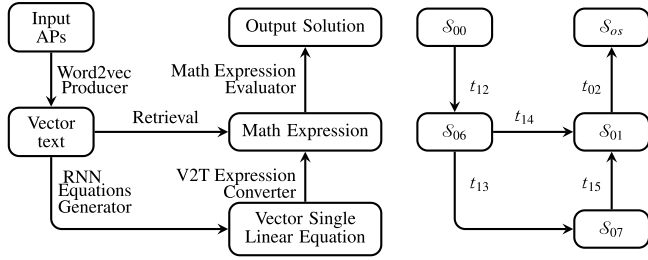


Fig. 1. Two representations of state transform graphs for $\mathcal{A}_{07}$: (a) using the terminology from the original paper, and (b) adopting the notations introduced in this paper.

The STA and the STT are highly related. The STT tells that solving algorithms have their own state transform graphs. This allows the algorithms to be uniformly compared and aggregated, highlighting their aggregate properties. In turn, the results from the STA validate the findings of the STT.

4.2. States and transforms

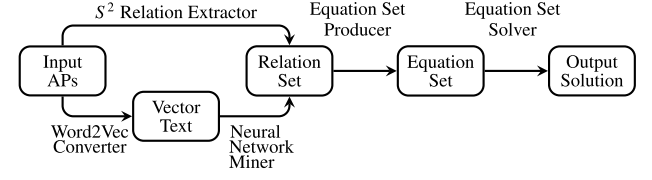
Two criteria are set for selecting the reference papers. This paper selects papers with two criteria. The first one is that they published since 2014 (or after 2013). The other one is that they developed decent algorithms for solving algebra problems narrated by word and diagrams. In other words, we do not select the papers that just discuss some methods without presenting the algorithm. In addition, we select the final journal paper as our representative paper if one algorithm is developed from the immature version in conference paper and the mature version in journal paper. With the above criteria, this paper gains 47 papers, thus defined 47 algorithms $\mathcal{A} = \{\mathcal{A}_{01}, \mathcal{A}_{02}, \dots, \mathcal{A}_{47}\}$, listed in Table 1. These 47 algorithms are divided into two categories of *original* and *incremental*. The algorithms in *original* are the first 30 algorithms in $\mathcal{A}$, proposing new states and/or new transforms or made the significant progress, say that the developed algorithms are published in high rank journals; the algorithms in *incremental* are the remaining 17 algorithms.

The 17 states are identified from $\mathcal{A} = \{\mathcal{A}_{01}, \mathcal{A}_{02}, \dots, \mathcal{A}_{30}\}$, denoted as $\Delta = \{S_{00}, S_{01}, S_{02}, \dots, S_{15}, S_{os}\}$. In Δ , S_{00} and S_{os} are two default states because any solving algorithms must have them. Table 1 lists the 15 non-default states and their basic facts. The items of the basic facts are reference paper (Within Table 1, we indicate reference papers by their serial numbers defined in the bottom of Table 1), development year, and occurrence number (the frequency with which the state is used in the 47 algorithms). We named each of the identified 17 states and described their original paper and features. Some further explanations are presented as follows.

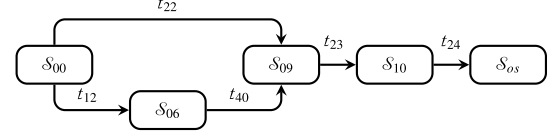
4.3. State transform graphs of individual algorithms

With the identified 17 states and 42 transforms from the 47 algorithms in the preceding listed in Table 1, this section will show that all the 47 algorithms can be depicted by the State Transform Graphs. Fig. 1 uses state transform graph to depict $\mathcal{A}_{07}$ proposed in Wang et al. (2017), being an example to draw the state transform graph of an algorithm.

Wang et al. (2017) proposed two methods, namely a seq2seq model and a similarity-based retrieval model to obtain expressions from Input APs. Thus, the algorithm in Wang et al. (2017) has two paths to obtain the expressions. Fig. 1(a) and (b) are the two versions of the state transform graph of $\mathcal{A}_{07}$, where 1(a) is the first version, which uses the terminologies in the original paper (Wang et al., 2017) and 1(b) is the second version, which uses the notations defined in this paper.



(a) Using the terms used in Yu et al. (2023)



(b) Using the defined notations

Fig. 2. Two representations of state transform graphs for $\mathcal{A}_{27}$: (a) using the terminology from the original paper, and (b) adopting the notations introduced in this paper.

Fig. 2 uses state transform graphs to depict $\mathcal{A}_{27}$ proposed in Yu et al. (2023), being another example to draw the state transform graph of an algorithm. Yu et al. (2023) proposed a relation-centric algorithm for solving arithmetic word problems, in which obtaining a set of relations as the understood state of the algorithm is the key task. The algorithm obtains the explicit and implicit relations separately. Hence, the algorithm in Yu et al. (2023) has two paths to obtain the relations. Fig. 2(a) and (b) are the two versions of the state transform graph of $\mathcal{A}_{27}$, where 2(a) is the first version, which uses the terminologies in the original paper (Yu et al., 2023) and 2(b) is the second version, which uses the notations defined in this paper.

From both the above examples, we can know that the state transform graphs for depicting individual algorithms can be a true graph, though most of such graphs may degrade into simple paths and that both algorithms can be depicted by state transform graphs, though they use the very different techniques.

4.4. Aggregated state transform graph

Fig. 1 shows that the algorithm proposed in Wang et al. (2017) is decomposable and can be abstracted into a graph with states as its nodes and transforms as its links. From the above analysis, we know that all the algorithms share the 17 states and 42 transforms. Based on these facts, the graphs of depicting a collection of algorithms can merge into an aggregated graph. Thus, we have a definition as follows.

Definition 8 (Aggregated Graph). Let $\mathcal{A}$ be a collection of algorithms for solving algebra problems. A graph $\mathcal{H}$ is called as an aggregated graph of $\mathcal{A}$ if it is formed by integrating all the graphs of depicting all individual algorithms in $\mathcal{A}$.

Fig. 3 is the aggregated graph of the algorithm set $\mathcal{A} = \{\mathcal{A}_{01}, \mathcal{A}_{02}, \dots, \mathcal{A}_{47}\}$. Fig. 3 includes all the states, all the transforms, and their links of all 47 algorithms for solving algebra problems in a graph. Hence, Fig. 3 reflects the overall picture of all the algorithms.

Notice: When some transforms can generate a linear equation set, of course they can generate a linear equation. However, Fig. 3 does not include the links that they can generate a linear equation for emphasizing their highest capabilities only.

4.5. State graph and approach

Since the aggregated graph in Figure 3 is for the collection of 47 algorithms, Fig. 3 can reflect the advance in developing algorithms for AP-solving. Hence, the conclusions about the global progress can be made by analyzing this graph. For example, we can focus on the reachability from one state to another without concerning the concrete transforms to conduct. For this objective, we can have the following state graph.

Table 1

The lists and the basic facts of the identified 47 algorithms, 15 non-default states, and 42 transforms for solving algebra problems from the identified 47 papers since 2014.

Table 1.1: The list of the identified 47 algorithms from the 47 identified papers and their basic facts													
Notation	$\mathcal{A}_{01}$	$\mathcal{A}_{02}$	$\mathcal{A}_{03}$	$\mathcal{A}_{04}$	$\mathcal{A}_{05}$	$\mathcal{A}_{06}$	$\mathcal{A}_{07}$	$\mathcal{A}_{08}$	$\mathcal{A}_{09}$	$\mathcal{A}_{10}$	$\mathcal{A}_{11}$	$\mathcal{A}_{12}$	$\mathcal{A}_{13}$
Year of Birth	2014	2014	2015	2015	2015	2015	2017	2018	2018	2019	2019	2019	2019
SN of Paper	[2]	[1]	[5]	[6]	[3]	[4]	[12]	[13]	[16]	[17]	[19]	[20]	[21]
# Created States	1	1	1	1	0	1	2	0	0	1	0	0	0
# Created Transforms	2	2	2	2	1	2	4	1	1	0	2	0	1
Notation	$\mathcal{A}_{14}$	$\mathcal{A}_{15}$	$\mathcal{A}_{16}$	$\mathcal{A}_{17}$	$\mathcal{A}_{18}$	$\mathcal{A}_{19}$	$\mathcal{A}_{20}$	$\mathcal{A}_{21}$	$\mathcal{A}_{22}$	$\mathcal{A}_{23}$	$\mathcal{A}_{24}$	$\mathcal{A}_{25}$	$\mathcal{A}_{26}$
Year of Birth	2019	2020	2020	2020	2020	2020	2021	2021	2021	2021	2021	2022	2022
SN of Paper	[22]	[23]	[28]	[25]	[24]	[26]	[29]	[30]	[34]	[31]	[37]	[41]	[42]
# Created States	0	2	0	1	0	0	1	0	1	0	0	2	0
# Created Transforms	1	3	0	2	1	0	2	1	2	1	1	4	1
Notation	$\mathcal{A}_{27}$	$\mathcal{A}_{28}$	$\mathcal{A}_{29}$	$\mathcal{A}_{30}$	$\mathcal{A}_{31}$	$\mathcal{A}_{32}$	$\mathcal{A}_{33}$	$\mathcal{A}_{34}$	$\mathcal{A}_{35}$	$\mathcal{A}_{36}$	$\mathcal{A}_{37}$	$\mathcal{A}_{38}$	$\mathcal{A}_{39}$
Year of Birth	2023	2023	2023	2023	2016	2016	2016	2017	2017	2018	2018	2019	2020
SN of Paper	[47]	[45]	[46]	[43]	[7]	[8]	[9]	[10]	[11]	[14]	[15]	[18]	[27]
# Created States	0	0	0	0	0	0	0	0	0	0	0	0	0
# Created Transforms	1	0	1	1	0	0	0	0	0	0	0	0	0
Notation	$\mathcal{A}_{40}$	$\mathcal{A}_{41}$	$\mathcal{A}_{42}$	$\mathcal{A}_{43}$	$\mathcal{A}_{44}$	$\mathcal{A}_{45}$	$\mathcal{A}_{46}$	$\mathcal{A}_{47}$					
Year of Birth	2021	2021	2021	2021	2022	2022	2022	2023					
SN of Paper	[40]	[44]	[33]	[32]	[36]	[35]	[38]	[39]					
# Created States	0	0	0	0	0	0	0	0					
# Created Transforms	0	0	0	0	0	0	0	0					

Table 1.2: The list of identified 15 non-default states from the first 30 identified algorithms and their basic facts

Notation	$\mathcal{S}_{01}$	$\mathcal{S}_{02}$	$\mathcal{S}_{03}$	$\mathcal{S}_{04}$	$\mathcal{S}_{05}$	$\mathcal{S}_{06}$	$\mathcal{S}_{07}$	$\mathcal{S}_{08}$	$\mathcal{S}_{09}$	$\mathcal{S}_{10}$	$\mathcal{S}_{11}$	$\mathcal{S}_{12}$	$\mathcal{S}_{13}$	$\mathcal{S}_{14}$	$\mathcal{S}_{15}$
Year of Birth	2014	2014	2015	2015	2015	2017	2017	2018	2019	2019	2020	2021	2021	2022	2022
Source Alg	$\mathcal{A}_{01}$	$\mathcal{A}_{02}$	$\mathcal{A}_{03}$	$\mathcal{A}_{06}$	$\mathcal{A}_{04}$	$\mathcal{A}_{07}$	$\mathcal{A}_{07}$	$\mathcal{A}_{10}$	$\mathcal{A}_{15}$	$\mathcal{A}_{15}$	$\mathcal{A}_{17}$	$\mathcal{A}_{20}$	$\mathcal{A}_{22}$	$\mathcal{A}_{25}$	$\mathcal{A}_{25}$
SN of Paper	[2]	[1]	[5]	[4]	[6]	[12]	[12]	[17]	[23]	[23]	[25]	[29]	[34]	[41]	[41]
# Occurrence	35	7	4	1	1	26	9	14	5	3	2	3	2	1	1

Table 1.3: The list of the identified 42 transforms from the first 30 identified algorithms and their basic facts

Notation	$\mathcal{T}_{01}$	$\mathcal{T}_{02}$	$\mathcal{T}_{03}$	$\mathcal{T}_{04}$	$\mathcal{T}_{05}$	$\mathcal{T}_{06}$	$\mathcal{T}_{07}$	$\mathcal{T}_{08}$	$\mathcal{T}_{09}$	$\mathcal{T}_{10}$	$\mathcal{T}_{11}$	$\mathcal{T}_{12}$	$\mathcal{T}_{13}$	$\mathcal{T}_{14}$	$\mathcal{T}_{15}$
Year of Birth	2014	2014	2014	2014	2015	2015	2015	2015	2015	2015	2015	2017	2017	2017	2017
SN of Paper	[2]	[2]	[1]	[1]	[5]	[5]	[4]	[4]	[3]	[6]	[6]	[12]	[12]	[12]	[12]
Start State	$\mathcal{S}_{00}$	$\mathcal{S}_{01}$	$\mathcal{S}_{00}$	$\mathcal{S}_{02}$	$\mathcal{S}_{00}$	$\mathcal{S}_{03}$	$\mathcal{S}_{00}$	$\mathcal{S}_{04}$	$\mathcal{S}_{00}$	$\mathcal{S}_{00}$	$\mathcal{S}_{05}$	$\mathcal{S}_{00}$	$\mathcal{S}_{06}$	$\mathcal{S}_{07}$	$\mathcal{S}_{06}$
Target state	$\mathcal{S}_{01}$	$\mathcal{S}_{05}$	$\mathcal{S}_{02}$	$\mathcal{S}_{05}$	$\mathcal{S}_{03}$	$\mathcal{S}_{01}$	$\mathcal{S}_{04}$	$\mathcal{S}_{03}$	$\mathcal{S}_{02}$	$\mathcal{S}_{05}$	$\mathcal{S}_{02}$	$\mathcal{S}_{06}$	$\mathcal{S}_{07}$	$\mathcal{S}_{01}$	$\mathcal{S}_{01}$
# Occurrence	3	32	3	7	3	6	1	1	1	1	1	26	6	3	8
Notation	$\mathcal{T}_{16}$	$\mathcal{T}_{17}$	$\mathcal{T}_{18}$	$\mathcal{T}_{19}$	$\mathcal{T}_{20}$	$\mathcal{T}_{21}$	$\mathcal{T}_{22}$	$\mathcal{T}_{23}$	$\mathcal{T}_{24}$	$\mathcal{T}_{25}$	$\mathcal{T}_{26}$	$\mathcal{T}_{27}$	$\mathcal{T}_{28}$	$\mathcal{T}_{29}$	$\mathcal{T}_{30}$
Year of Birth	2018	2018	2019	2019	2019	2019	2019	2019	2019	2020	2020	2020	2021	2021	2021
SN of Paper	[13]	[16]	[19]	[19]	[21]	[22]	[23]	[23]	[23]	[24]	[25]	[25]	[29]	[29]	[30]
Start State	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{08}$	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{09}$	$\mathcal{S}_{10}$	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{11}$	$\mathcal{S}_{06}$	$\mathcal{S}_{12}$	$\mathcal{S}_{06}$
Target state	$\mathcal{S}_{07}$	$\mathcal{S}_{08}$	$\mathcal{S}_{08}$	$\mathcal{S}_{01}$	$\mathcal{S}_{08}$	$\mathcal{S}_{08}$	$\mathcal{S}_{09}$	$\mathcal{S}_{10}$	$\mathcal{S}_{05}$	$\mathcal{S}_{07}$	$\mathcal{S}_{11}$	$\mathcal{S}_{08}$	$\mathcal{S}_{12}$	$\mathcal{S}_{02}$	$\mathcal{S}_{13}$
# Occurrence	1	1	1	12	1	1	1	3	3	3	4	8	1	1	1
Notation	$\mathcal{T}_{31}$	$\mathcal{T}_{32}$	$\mathcal{T}_{33}$	$\mathcal{T}_{34}$	$\mathcal{T}_{35}$	$\mathcal{T}_{36}$	$\mathcal{T}_{37}$	$\mathcal{T}_{38}$	$\mathcal{T}_{39}$	$\mathcal{T}_{40}$	$\mathcal{T}_{41}$	$\mathcal{T}_{42}$			
Year of Birth	2021	2021	2021	2021	2022	2022	2022	2022	2022	2023	2023	2023			
SN of Paper	[31]	[34]	[34]	[37]	[41]	[41]	[41]	[41]	[42]	[47]	[46]	[43]			
Start State	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{13}$	$\mathcal{S}_{06}$	$\mathcal{S}_{00}$	$\mathcal{S}_{00}$	$\mathcal{S}_{14}$	$\mathcal{S}_{15}$	$\mathcal{S}_{06}$	$\mathcal{S}_{06}$	$\mathcal{S}_{00}$	$\mathcal{S}_{00}$			
Target state	$\mathcal{S}_{09}$	$\mathcal{S}_{13}$	$\mathcal{S}_{03}$	$\mathcal{S}_{02}$	$\mathcal{S}_{14}$	$\mathcal{S}_{14}$	$\mathcal{S}_{15}$	$\mathcal{S}_{05}$	$\mathcal{S}_{08}$	$\mathcal{S}_{09}$	$\mathcal{S}_{09}$	$\mathcal{S}_{09}$			
# Occurrence	3	1	1	1	1	1	1	1	1	3	1	1			

The notations from [1] to [47] appeared in this table are defined as follows: [1]: Kushman et al. (2014) [2]: Hosseini et al. (2014) [3]: Zhou et al. (2015) [4]: Koncel-Kedziorski et al. (2015) [5]: Roy and Roth (2015) [6]: Shi et al. (2015) [7]: Mitra and Baral (2016) [8]: Roy et al. (2016) [9]: Upadhyay et al. (2016) [10]: Huang et al. (2017) [11]: Ling et al. (2017) [12]: Wang et al. (2017) [13]: Huang et al. (2018a) [14]: Huang et al. (2018b) [15]: Roy and Roth (2018) [16]: Wang et al. (2018b) [17]: Wang et al. (2018a) [18]: Li et al. (2019) [19]: Liu et al. (2019) [20]: Rehman et al. (2019) [21]: Wang et al. (2019) [22]: Xie and Sun (2019) [23]: Yu et al. (2019) [24]: Kim et al. (2020) [25]: Li et al. (2020) [26]: Mandal et al. (2020) [27]: Wu et al. (2020) [28]: Zhang et al. (2020b) [29]: Cao et al. (2021) [30]: Lin et al. (2021) [31]: Lyu and Yu (2021) [32]: Mandal and Naskar (2021) [33]: Qin et al. (2021) [34]: Tsai et al. (2021) [35]: Wu et al. (2021b) [36]: Wu et al. (2021a) [37]: Zaporjets et al. (2021) [38]: Jie et al. (2022) [39]: Li et al. (2022) [40]: Liang et al. (2022) [41]: Yu et al. (2022) [42]: Zhang et al. (2022) [43]: Huang et al. (2023) [44]: Liang et al. (2023) [45]: Lin et al. (2023) [46]: Meng et al. (2023) [47]: Yu et al. (2023)

Definition 9 (State Graph). Let $\mathcal{H}$ be an aggregated graph. $\mathcal{H}'$ is called as the state graph of $\mathcal{H}$ if it is formed by converting the multiple links of $\mathcal{H}$ that link the same pair of states into single link.

From the above definition, state graph concerns that whether there is a link to link a pair of states. In other word, it pays attention to the reachability of states. The state graph in Fig. 4 is formed from Fig. 3. From Fig. 4, approach can be defined as follows.

Definition 10 (Understood State). A state is an understood state if a symbolic solver can produce the solution from the instance of the state of any AP.

From Fig. 4, four understood states are identified; they are math expression, linear equations, general equations, and xEquations.

Definition 11 (Approach). An approach of solving algebra problems is the partial graph of the state graph that includes all the simple paths of passing the concern understood state.

From the above definition, an approach is an abstract description of the algorithms of sharing the same understood state, and an algorithm is an instance of an approach. In addition, the factors that can reflect the difference of algorithms are its states, followed by its transforms. According to Definition 11, four approaches are identified and

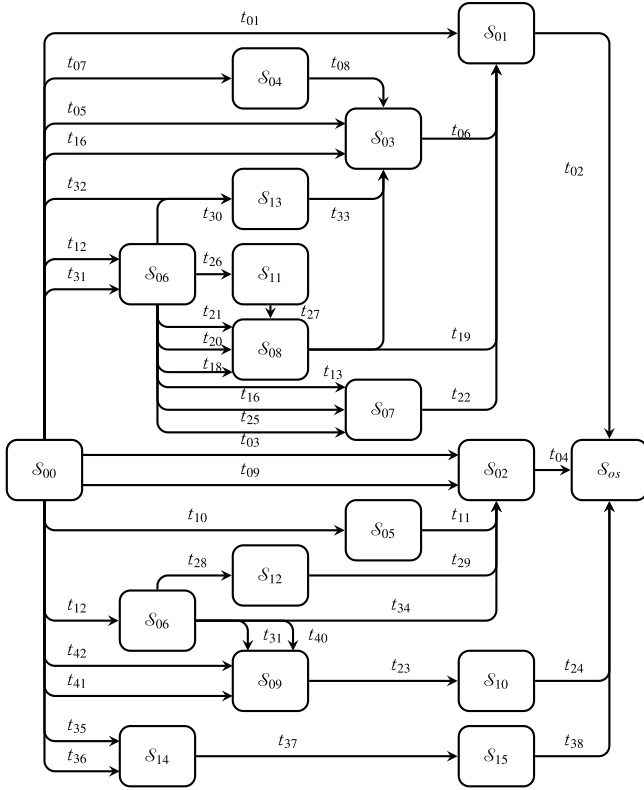


Fig. 3. The aggregated graph of the identified 47 algorithms.

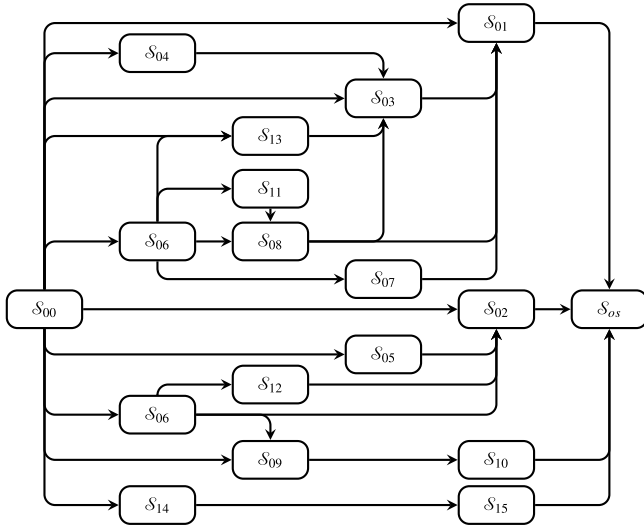


Fig. 4. The state graph of the algorithm set $\mathcal{A} = \{\mathcal{A}_{01}, \mathcal{A}_{02}, \dots, \mathcal{A}_{47}\}$.

listed in Fig. 5. They are Ω_1 = Math Expression, Ω_2 = Linear Equations, Ω_3 = Relation-Centric, Ω_4 = xRelation-Centric.

4.6. Taxonomy of algorithms

Appendix B presents the state transform graphs for each of the 47 individual algorithms. In Section 4.4, these individual graphs are combined into an aggregate state transform graph, which is further abstracted into a state graph, and four distinct approaches are defined. These analyses and results offer deeper insights into the problem-solving algorithms.

Building on this improved understanding, a new taxonomy for the algorithms is introduced, organized as a two-level hierarchy. At the first

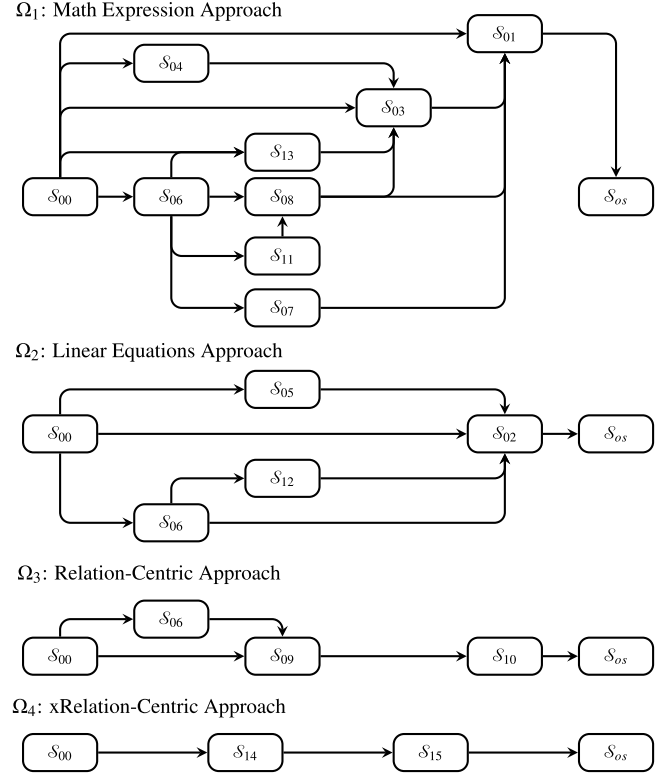


Fig. 5. Four graphs of depicting the identified four approaches from Fig. 4.

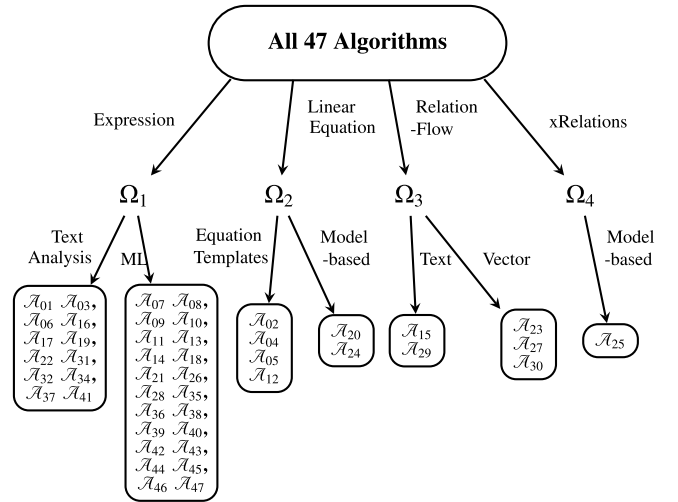


Fig. 6. Two-level taxonomy of the 47 identified algorithms for solving algebraic problems.

level, the algorithms are grouped into four overarching approaches. At the second level, they are further categorized based on the primary techniques they employ to solve problems. This taxonomy is illustrated in Fig. 6.

5. Evolution of four approaches

For the four approaches defined earlier, this section will delve into the detailed establishment and progression of each approach. The focus will be on examining the development of these approaches over time, including both the evolution of their states and their transforms.

5.1. Math expression approach

The first approach denoted as Ω_1 and its characteristics is to acquire a math expression that can produce the value of the unknown of the problem by evaluating it. Hence, the algorithms belong this approach can solve only the problems with single unknown.

5.1.1. Ω_1 state evolution

The essential task of Ω_1 is to accomplish the transform from Input AP ($\mathcal{S}_{00}$) to Math Expression ($\mathcal{S}_{01}$) for the given problem. Since this task is hard to be done by a single transform, researchers proposed some bridge states.

Roy and Roth (2015) proposed Expression Tree ($\mathcal{S}_{03}$) as the first bridge state. Koncel-Kedziorski et al. (2015) continued the idea of using Expression Trees ($\mathcal{S}_{04}$) to bridge Input AP ($\mathcal{S}_{00}$) and Expression Tree ($\mathcal{S}_{03}$). Expression Trees ($\mathcal{S}_{04}$) are the set of the possible expression trees, which is prepared to identify the wanted expression tree.

Inspired by the success of seq2seq translation, Wang et al. (2017) first brought seq2seq method into solving APs. Hence, two states, vector text ($\mathcal{S}_{06}$) and vector single linear equation ($\mathcal{S}_{07}$), were proposed as bridge states. Since then, various deep neural networks were proposed to translate $\mathcal{S}_{06}$ into $\mathcal{S}_{07}$ (Mandal et al., 2020; Wang et al., 2019). Wang et al. (2018a) used vector computing to translate from $\mathcal{S}_{06}$ into vector expression tree $\mathcal{S}_{08}$ instead of $\mathcal{S}_{07}$. Since vector expression tree can keep the structure information, more transforms were developed to transform from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$ (Jie et al., 2022; Lin et al., 2021; Liu et al., 2019; Wang et al., 2019; Wu et al., 2021a,b; Xie & Sun, 2019; Zaporozhets et al., 2021; Zhang et al., 2022).

Inspired by the better performance by adding structural information in expressions Wang et al. (2019), Liu et al. (2019), Xie and Sun (2019), Zhang et al. (2020b) firstly proposed Quantity Cell Graph and Quantity Comparison Graph to enrich the quantity representations by capturing the relation and order structure between quantities. Li et al. (2020) combined the word sequence with the corresponding dependency or constituency tree to enrich input information. These expressions, which combine graphs and problem text, are named as TextGraph ($\mathcal{S}_{11}$) and is the extended state of $\mathcal{S}_{06}$.

5.1.2. Ω_1 transform evolution

In the early years, the researchers developed the multiple transforms from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$. With the observation that verbs may determine the operands in the expression of a given AP, Hosseini et al. (2014) developed a transform, being denoted as t_{01} . They presented a verb categorization based method, namely ARIS to solve simple addition and subtraction APs. ARIS first analyzes each of the sentences in the problem to identify the relevant variables and their values, and then mapped this information into a predefined logic template, namely “container” so as to capture the relations between variables. A verb in a sentence is classified into one of seven categories to determine the operator between two quantities. Mitra and Baral (2016) adopted a similar idea, first using predefined logical templates to capture the types of relations between variables, thereby generating expressions. Although these methods perform well on some datasets, they are considered unable to be popularized on large datasets due to the high cost of data annotation.

Taking Expression Tree ($\mathcal{S}_{03}$) as a bridge state, Roy and Roth (2015) formalized the AP-solving problem into multiple classification problems, and generated the expression by judging whether the number in the expression participated in the composition of the expression and the operator between the two candidate numbers. More specifically, they proposed t_{05} to generate Expression Tree ($\mathcal{S}_{03}$) from Input APs ($\mathcal{S}_{00}$), and t_{06} to generate Math Expression ($\mathcal{S}_{01}$) from Expression Tree ($\mathcal{S}_{03}$). Koncel-Kedziorski et al. (2015) took the Expression Trees ($\mathcal{S}_{04}$) as a bridge state between $\mathcal{S}_{01}$ and $\mathcal{S}_{03}$ and proposed t_{07} , namely integer linear programming to generate candidate expression trees, and t_{08} to identify a wanted

expression tree by scoring the candidate expression trees through local and global discriminant models and selecting the one with the highest score.

Compared with the algorithm in Hosseini et al. (2014), the ones in Roy and Roth (2015) and Koncel-Kedziorski et al. (2015) have two progresses. First, they are machine learning based so that they are compact without requiring tedious processes such as annotation, classification. Second, they have a higher accuracy. However, due to the limited expression ability of the state $\mathcal{S}_{03}$, they can solve only problems with single unknown.

Inspired by the recent advancements in NLP (Natural Language Processing), the seq2x approach is developed (seq2x is the umbrella name of seq2seq, seq2tree, seq2graph, etc). This approach targets to obtain the expression that can produce the answer by various neural networks. It first transforms the text into the vector text by an encoder and then decodes vector text into vector expression, or vector expression tree, or vector graph.

DNS (Wang et al., 2017) is the first work to use seq2seq model for solving APs, which aims to directly generate Vector Single Linear Equation ($\mathcal{S}_{07}$) from Vector Text ($\mathcal{S}_{06}$), namely the input and the output are the sequence of Vector Text ($\mathcal{S}_{06}$) and the sequence of Vector Single Linear Equation ($\mathcal{S}_{07}$). It adopted the gated recurrent units (GRU) to convert $\mathcal{S}_{06}$ to $\mathcal{S}_{07}$, and the long short-memory (LSTM) to convert $\mathcal{S}_{07}$ to $\mathcal{S}_{02}$. Wang et al. (2018a) followed the idea of DNS (Wang et al., 2017), and proposed equation normalization to solve the problem of equation duplication in DNS (Wang et al., 2017) by generated an expression tree. Huang et al. (2018a) adjusted the output-sequence generation process by copy and alignment mechanism (namely CASS) to the seq2seq model, and solved the problem of the false numbers in the wrong position in the seq2seq model. In Huang et al. (2018a), they proposed t_{16} to generate Vector Single Linear Equation ($\mathcal{S}_{07}$) from Vector Text ($\mathcal{S}_{06}$) using the reinforcement learning (RL).

Wang et al. (2018a,b) showed the important contribution of structured information in equations in improving the performance of solvers. A new state of Vector Expression Tree ($\mathcal{S}_{08}$) is proposed. With the bridge states of Vector Text ($\mathcal{S}_{06}$) and Vector Expression Tree ($\mathcal{S}_{08}$), some works (Liu et al., 2019; Wang et al., 2019; Xie & Sun, 2019) used the structural information of the expression to improve the effect of the seq2seq models. Liu et al. (2019) proposed t_{18} to generate $\mathcal{S}_{08}$ from $\mathcal{S}_{06}$, and t_{19} to transform $\mathcal{S}_{06}$ to $\mathcal{S}_{01}$. Wang et al. (2019) proposed t_{20} to generate $\mathcal{S}_{08}$ from $\mathcal{S}_{06}$. Xie and Sun (2019) proposed t_{21} to explicitly generate $\mathcal{S}_{08}$ from $\mathcal{S}_{06}$ in a human-like goal-driven way. So it could be seen as an expansion of the work of Wang et al. (2018a), and is the first tree-structured neural model for APs.

The problem text also contain some structure information, which could contribute to improve the performance of the solving algorithms (Huang et al., 2018b; Wang et al., 2018a). Inspired by this, a new bridge state of TextGraph ($\mathcal{S}_{11}$) was proposed. With this bridge state, Zhang et al. (2020b) firstly adopted the graph2tree model for AP-solving which is a novel deep learning architecture that combines the merits of the graph-based encoder and tree-based decoder to generate better solution expressions. Li et al. (2020) proposed a novel Graph2Tree Neural Network, consisting of a graph encoder and a hierarchical tree decoder, that encodes an augmented graph-structured input and decodes a tree-structured output. Both of them claimed that their models outperform the state-of-the-art baselines on benchmark datasets, proving the superiority of $\mathcal{S}_{11}$ and graph2Tree models in AP-solving.

5.2. Linear equations approach

This approach is denoted as Ω_2 , targeting to acquire a set of linear equations to represent the given problem. Hence, this approach can solve the problems with multiple unknowns. The goal of this approach is to generate Linear Equations ($\mathcal{S}_{02}$) from the Input APs ($\mathcal{S}_{00}$).

5.2.1. Ω_2 state evolution

Kushman et al. (2014) first built this approach. The algorithm in Kushman et al. (2014) used a state of Linear Equations ($\mathcal{S}_{02}$) to represent the given problem. It used a template based method to obtain the system of linear equations. The algorithm has a set of templates of abstract linear equations; it used a machine learning based procedure to select a template and to instantiate with the quantity entities from the given problem.

Shi et al. (2015) first proposed “Formal Language” ($\mathcal{S}_{05}$) as the first bridge state. “Formal Language” appeared in a form of tree, whose each internal node is a function, and each leaf node can be a constant, a class, or a zero-argument function.

The previous seq2tree models have proved the importance of structural information. However, seq2tree cannot handle multiple unknown problems. Because the equations in the equation set problem are unordered, but for the seq2tree and seq2seq models, the generation of equations will involve the order problems, which will add unnecessary burden to the model. To solve this problem, Cao et al. (2021) firstly proposed $\mathcal{S}_{12}$ (DAG = Direct Acyclic Graph) as the new bridge state, where leaf nodes of DAG are the quantities, and other (internal and root) nodes are arithmetic or comparison operators. It integrates multiple expression trees of an equation set problem into the same solving scenario, so as to obtain a DAG. In this way, it not only preserves the structural information of the expression, but also extends the scope to the equation set problems.

5.2.2. Ω_2 transform evolution

Kushman et al. (2014) proposed a template-based transform (t_{03}), which identifies a template from the corpus of predefined templates and then fills the number slots and unknown slots with the quantity entities extracted from the text to produce the instances of $\mathcal{S}_{02}$. It also proposed to use t_{04} to solve $\mathcal{S}_{02}$ to get $\mathcal{S}_{05}$. In order to reduce the hypothesis space, Zhou et al. (2015) improved the work of Kushman et al. (2014) by only considering assigning the number slots. They developed another transform (t_{09}), which is a log-linear model, to directly transform $\mathcal{S}_{00}$ into $\mathcal{S}_{02}$.

With the bridge state of “Formal Language” ($\mathcal{S}_{05}$), Shi et al. (2015) firstly proposed a context-free grammar parser (t_{10}) to translate Input APs ($\mathcal{S}_{00}$) into $\mathcal{S}_{05}$, and a reasoning module (t_{02}) to derive Linear Equation Set ($\mathcal{S}_{02}$) from $\mathcal{S}_{05}$. This method uses the symbolic reasoning to obtain equations, making enhancement in evaluation and parsing.

With the bridge state of $\mathcal{S}_{12}$ (DAG), Cao et al. (2021) proposed a Seq2DAG model for AP-solving. This paper proposed DAG Extraction Decoder (t_{28}) to transform Vector Text of Input AP directly into a DAG structure, and t_{29} to generate Linear Equation Sets from DAG. These transforms together extended the solving scope by including the problems with multiple unknowns because the system of linear equations can find the values of multiple unknowns.

5.3. Relation-centric approach

This approach Ω_3 was first proposed by Yu et al. (2019). This approach is called as relation-centric approach because its main actions are to acquire Relation Set ($\mathcal{S}_{09}$) and to transform $\mathcal{S}_{09}$ to obtain Equation Set ($\mathcal{S}_{10}$), and then to find the Output Solution ($\mathcal{S}_{05}$), in which relations are atomic operands of transforms. The relation-centric approach is comparable with Math Expression approach (Ω_1) and Linear Equations approach (Ω_2), but it is different from them in two aspects. First, the equations converted from the relations can be high-order equations. Such high-order equation set can represent the larger scope of algebra problems than the set of linear equations can do. Second, acquiring relations is easier than acquiring equations, although the processes of acquiring relations and acquiring equations have just little difference in appearance. On the other hand, it is relatively easy to convert relations into equations, but it is hard to reverse equations into relations.

5.3.1. Ω_3 state evolution

On relation-centric approach (Ω_3), the initial algorithms have two exclusive states $\mathcal{S}_{09}$ to $\mathcal{S}_{10}$ in Yu et al. (2019). Inspired by the success of vector computing in NLP, Lyu et al. (2023) introduced vector computing technology into relation-centric approach. Thus, Vector Text ($\mathcal{S}_{06}$) was brought into this approach, first proposed in Wang et al. (2017) in 2017.

5.3.2. Ω_3 transform evolution

Yu et al. (2019) proposed a relation-centric approach and constructed an algorithm with t_{22} (S^2 Relation Extractor), t_{23} (Equation Set Producer), and t_{24} (Equation Set Solver). In 2023, Lyu et al. (2023) evolved t_{22} into t_{31} , a new transform that uses the vector S^2 method to extract $\mathcal{S}_{09}$ from $\mathcal{S}_{06}$ of APs. Lyu et al. (2023) is the updated version of Lyu and Yu (2021).

Acquiring implicit relations from algebra problems is an indispensable problem because solving problems requires to use implicit relations (Yu et al., 2019). On the other hand, acquiring implicit relations from algebra problems is a challenge problem because students use various ways to acquire them when they do homework. Yu et al. (2023) developed t_{40} (Neural Network Miner), a neural network based transform, mining implicit relations from $\mathcal{S}_{06}$ (Vector Text). Yu et al. (2023) developed a high-performance algorithm for solving arithmetic word problems by synergizing syntax-semantics extractor for explicit relations and neural network miner for implicit relations. Lyu et al. (2023) enhanced the algorithm developed in Yu et al. (2023) in multiple aspects by using vector syntax-semantics extractor. Meng et al. (2023) developed t_{42} (DI Relation Extractor, which extracts Deep Implicit relations), a new relation extractor that can extract DI relations from problems text. Huang et al. (2023) developed t_{41} (GD Relation Extractor, which extracts relations from diagrams), a new relation extractor that can extract GD relations from Geometry Diagrams. Both t_{41} and t_{42} extend the scope of solving algorithms.

5.3.3. Ω_3 application in solving other types of problems

Ω_3 has also applied to solve other types of problems. Gan et al. (2019a,b) have successfully used S^2 relation extractor in Ω_3 to understand plane geometry theorems. Jian et al. (2019) and He et al. (2020) have successfully used S^2 relation extractor in Ω_3 to understand linear circuit problems.

5.4. xRelation-centric approach

5.4.1. Ω_4 state evolution

Yu et al. (2022) developed a new approach Ω_4 (xRelation-Centric Approach). Ω_4 is the first approach that has mechanism to handle the compound object of functions. The algorithm in Yu et al. (2022) is the first and only algorithm in this approach so far. The algorithm has two new states: xRelation Set ($\mathcal{S}_{14}$) and xEquation Set ($\mathcal{S}_{14}$). xRelation set consists of universal relations and period relations; each clique of period relations potentially describes a function. xEquation Set consists of high-order equations and functions.

5.4.2. Ω_4 transform evolution

The algorithms in Yu et al. (2022) has four new transforms. The first transform is t_{35} (S^2P Relation Extractor), extracting period relations from text of function problems. The second transform is t_{36} (L^2 Relation Extractor), acquiring relations from diagrams of algebra problems by a line segment and label method, shorted as L^2 method. The third transform is t_{37} (xEquation Set Producer), converting $\mathcal{S}_{14}$ (xRelation Set) into $\mathcal{S}_{14}$ (xEquation Set). The fourth transform is t_{38} (xEquation Set Solver), obtaining the answers of function problems. This transform is required to interact between equations and functions to find the answers.

6. Perspective confusion comparison

The algorithms in the literature were developed to address various types of APs, each of which presents distinct challenges. To gain a per-

formance overview of the existing algorithms, the types of problems are defined, and a comparative evaluation is performed to assess the transforms, algorithms, and approaches across these problem types. However, this section is not intended to provide an exhaustive comparative analysis. Instead, it aims to verify the feasibility of the new evaluation paradigm, given the specific focus of this paper.

Definition 12 (Perspective Confusion Comparison). The Perspective Confusion Comparison (PCC) is an evaluation paradigm for comparing a set of algorithms designed to solve a complex problem that its instances can be classified into different types. This paradigm assumes that algorithms can be represented as state transform graphs. The PCC involves conducting confusion experiments, where each transform, each algorithm, and each approach are evaluated across various problem types to assess their performance and applicability.

6.1. Types of algebra problems

The types of problems are defined according to 3 features and 2 modalities, being defined below. Each problem will express some conditions so that solvers solve it by using these conditions. Problems may state some of these conditions explicitly; it also may tell some conditions implicitly. Hence, we have the following definition.

Definition 13 (Explicit/Implicit Condition). When solving an algebra problem, some conditions are needed. A condition is called as an explicit condition if the problem explicitly states it; otherwise, it is an implicit condition. Further, an implicit condition is called as shallow implicit condition if some expression of the problem can hint it; otherwise, it is a deep implicit condition.

The current indicator is tentatively used to differ shallow implicit conditions from deep implicit conditions because this indicator is not perfect yet. According to which features the considered problem possesses, whether it has diagrams, and the non-double inclusion principle, the six types of algebra problems are defined. The following non-double inclusion principle is introduced to avoid the occurrence that one problem is divided into two types.

Non-double inclusion principle: Type j do not contain $\mathcal{P}$ if Type i has contained $\mathcal{P}$ and $i < j$.

The algorithms will be analyzed on three datasets: M23K, TnD1K and DRAW-1K. M23K is from Math23K (Wang et al., 2017) by discarding pure numerical problems and special symbol calculation, which contains 20910 algebra word problems with only one variable (Yu et al., 2023). TnD1K contains 1000 function problems (Yu et al., 2022). DRAW-1K contains 747 algebra word problems with two unknowns and 255 algebra word problems with one unknown (Kushman et al., 2014). Table 2 shows: 1) all the three datasets do not include the problems of Type 5 and 6; 2) M23K and DRAW-1K, two of the representative popular datasets, do not include the problems of Type 4; 3) the distributions of these three datasets over Type 1 to 4 are presented. The next section will analyze algorithms over four types of problems.

6.2. Reachability and efficiency of transforms

Some algorithms and transforms are selected to demonstrate the perspective confusion comparison. The selected algorithms are $\mathcal{A}_{07}$, $\mathcal{A}_{20}$, $\mathcal{A}_{21}$, $\mathcal{A}_{25}$, $\mathcal{A}_{27}$, $\mathcal{A}_{29}$, $\mathcal{A}_{30}$ and $\mathcal{A}_{35}$, where $\mathcal{A}_{07}$, $\mathcal{A}_{21}$, and $\mathcal{A}_{35}$ belong to Math Expression (Ω_3), $\mathcal{A}_{20}$ belongs to the Linear Equations, $\mathcal{A}_{27}$, $\mathcal{A}_{29}$, $\mathcal{A}_{30}$ belong to Relation Centric, and $\mathcal{A}_{25}$ belongs to the xRelation-Centric approach, respectively. The transform efficiency are define as follows.

Definition 14 (Transform Efficiency). Let $\mathcal{C}$ be a corpus and $t(i, j)$ be a transform from $\mathcal{S}_i$ to $\mathcal{S}_j$ in solving algebra problems. The transform efficiency is defined as the accuracy that $t(i, j)$ transforms all $\mathcal{S}_i$ instances of $\mathcal{C}$ into $\mathcal{S}_j$.

It is easy to define the transform efficiency, but it is hard to obtain it. The difficulty of obtaining the transform efficiency lies in that it is hard

Table 2

The definitions of 6 types of algebra problems and the distribution of datasets M23K, TnD1K, DRAW-1K, DIR-AWPs, and GCP-PS over Type 1 to 5.

Table	Modality		Three Features		
	Text	Diagram	Explicit	Shallow	Deep
Type 1	✓		✓		
Type 2	✓		✓	✓	
Type 3	✓		✓	✓	✓
Type 4	✓	✓	✓		
Type 5	✓	✓	✓	✓	
Type 6	✓	✓	✓	✓	✓

Dataset	Distributions over Type 1 to 5				
	Type 1	Type 2	Type 3	Type 4	Type 5
M23K	16568	3591	741	0	0
TnD1K	530	74	0	395	0
DRAW-1K	677	148	174	0	0
DIR-AWPs	0	0	6030	0	0
GCP-PS	0	0	0	0	217

Note: All the datasets in this table do not include the problems in Type 6.

to know the correctness of instances of its start state and target state for some transforms. Especially for deep learning based algorithms, the instances in vector are difficult in knowing their correctness. Maybe, that is the reason that the research community requires only the ablation experiments.

Since it is hard to find the transform efficiency, we turn to find the efficiencies of the partial algorithms as an substitute. Let $\mathcal{E}_1$ be the efficiency of the partial graph of the state transform graph (STG) of $\mathcal{A}_{35}$ in Appendix B from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$ that includes t_{05} and t_{06} ; let $\mathcal{E}_2$ be the efficiency of the partial graph of the STG of $\mathcal{A}_{07}$ in Appendix B from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$ that includes t_{12} , t_{13} and t_{15} ; let $\mathcal{E}_3$ be the efficiency of the partial graph of the STG of $\mathcal{A}_{21}$ in Appendix B from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$ that includes t_{12} , t_{30} and t_{06} ; let $\mathcal{E}_4$ be the efficiency of the partial graph of the STG of $\mathcal{A}_{20}$ in Appendix B from $\mathcal{S}_{00}$ to $\mathcal{S}_{02}$ that includes t_{12} , t_{28} and t_{29} . Because we can know the correctness of $\mathcal{S}_{02}$ and $\mathcal{S}_{01}$, we can calculate $\mathcal{E}_1$ and $\mathcal{E}_2$.

Table 3 shows the perspective confusion comparison of 8 algorithms, 4 partial algorithms and 4 transforms from 4 approaches tested on 5-type distribution of 5 datasets of algebra problems. In Table 3 the data with superscript stars are from the corresponding published papers; the other data are produced by this paper.

Table 3 reveals the following findings. First, the algorithm $\mathcal{A}_{30}$ presented in Huang et al. (2023) is the only algorithm that can solve problems in Type 5 and it achieved an accuracy of 81.5 % on a dataset named as GCP-PS. Second, the xRelation-Centric approach can solve multiple types of problems. The algorithms $\mathcal{A}_{25}$ presented in Yu et al. (2022) are the only algorithms for solving the problems with diagrams (namely Type 4) so far. Third, only the algorithm $\mathcal{A}_{25}$ presented in Meng et al. (2023) has achieved an accuracy of 84.5 % on a dataset named as DIR-AWPs, while all other existing algorithms have relatively low accuracy in solving problems in Type 3. Thus, it is the potential research problem to develop methods to acquire the shallow implicit relations to improve the algorithm accuracy for solving APs. Fourth, $\mathcal{A}_{31}$ also reports the accuracy of only 26.2 % over Type 2, though it can solve more types of APs. It shows that $\mathcal{A}_{31}$ does not adopt the best techniques in the existing algorithms.

Compared with one-level confusion comparison and ablation experiment, the perspective confusion comparison (PCC) has the following features:

1) The PCC can evaluate not only the external indicators of algorithms but also their internal indicators such as the transform efficiency over the various types of problems. Thus, it can reveal the advantages and disadvantages of transforms, algorithms and approaches on the various types of problems.

2) Compared with the ablation experiment and one-level confusion evaluation, the PCC can reveal more research gaps. The results of the

PCC are three dimension space covering the results of the competitive comparison as its partial space. Hence, the PCC can show more research gaps because any point of the three dimension space has potential to have research gaps.

7. Conclusions and research directions

This paper has done a theoretical review on solving algebra problems, which has built the state transform theory (STT), performed the state transform analysis (STA) of individual algorithms and collective algorithms, and proposed the PCC for solving algebra problems. Through this review, we have lead the five conclusions presented in Section 7.1 and we can propose the six research directions presented in Section 7.2.

7.1. Conclusions and discussions

This paper has developed a theoretical review framework for reviewing the algorithms for solving algebra problems, which are published since 2014 (or after 2013). This framework consists of State Transform theory, State Transform Analysis method for structure analysis of individual and collective algorithms, state/transform approach review method and perspective confusion comparison scheme. The main conclusions on the above mentioned five aspects are presented as follows.

7.1.1. Theoretical review framework

This paper has developed the theoretical review framework for reviewing algorithms for solving algebra problems since 2014. This framework consists of STT, STA, and PCC. This framework differs from the traditional review way in three aspects. First, it reviews algorithms based on the STT. Second, it uses the various state transform graphs to depict individual algorithms and collective algorithms; it can uniformly define approaches on the state graph, instead of the traditional way, i.e. defining approaches based on the hand-craft indicators. Third, it extends the traditional algorithm comparison scope into a perspective confusion comparison for transform, algorithm, and approach. Thus, it reveals not only the competitive performance among algorithms but also the internal contributions of algorithms (transform contributions) and the performance of approaches.

7.1.2. State transform theory

This paper introduces a State Transform Theory (STT), which highlights the crucial roles of states and transforms in solving algorithms. The significance of this theory lies in its distinction between two capacities: the theoretical capacity, determined by the structure of state transforms, and the technical capacity, governed by the specific transforms used in algorithms. According to this theory, the evolution of both states and transforms defines the progress of solving algorithms, whereas traditional surveys and reviews have primarily focused on the progression of transforms (techniques). The core assertions of the STT are as follows:

1) Most algorithms for solving algebraic problems can be represented as a state transform graph; 2) Representative algorithms developed since 2014 are decomposable; 3) All representative algorithms for solving algebraic problems developed since 2014 are constructed using 17 states and 42 transforms.

Earlier review frameworks also support the STT. The first review paper on solving algorithms already marks the emergence of the STT. In that paper, the two-stage review framework is considered the beginning of the STT. This framework is based on the cognition that solving algorithms involve two stages: problem understanding and symbolic solver. The outcome of the first stage is essentially the understood representation within the STT. Notably, most review papers adopt this two-stage framework, even if implicitly. These papers focus on the advancement of techniques, which are, in essence, the implementation methods for the transforms that are more abstract than the techniques themselves.

7.1.3. State transform analysis

This paper has developed the State Transform Analysis (STA) based on the STT. In this method, each algorithm can be depicted by a state transform graph, a collection of algorithms can be depicted by an aggregated graph, and the state structures of a collection of algorithms can be depicted by a state graph. Furthermore, the approaches can be uniformly defined on the state graph according to the understood states. Such analysis identified 17 states, 4 understood states and 42 transforms for the solving algorithms proposed since 2014. The 4 understood states further defined 4 approaches. The traditional review or survey papers emphasized on reviewing the techniques of algorithms whereas this paper emphasized on both states and transforms, which is the umbrella terms of the techniques.

7.1.4. State transform review scheme

This paper has built a state/transform review scheme. This scheme reviews an approach based on its state progress and transform progress. The traditional review papers emphasized on the transforms of approaches whereas this paper emphasized on both states and transforms.

7.1.5. Perspective confusion comparison

This paper introduces the Perspective Confusion Comparison (PCC) framework based on state transform theory. The proposed scheme extends traditional one-level confusion comparison into a three-level structure, encompassing transforms, algorithms, and approaches. The PCC systematically compares these elements across various problem types, offering a more comprehensive understanding of the knowledge framework underlying solving algorithms. Unlike ablation experiments, the PCC not only evaluates performance across multiple levels but also provides a more granular analysis of specific problem types.

The PCC offers several advantages over ablation experiments when evaluating solving algorithms. First, the PCC captures all experimental

Table 3

Confusion comparison results of 8 algorithms, 4 partial algorithms and 4 transforms from 4 approaches over 5 types of problems of 5 datasets.

Ω_s	$\mathcal{A}_s$	Source paper	Dataset	Algorithm accuracy on 5 types (%)					Transform	Transform efficiency on 5 types (%)				
				Type 1	Type 2	Type 3	Type 4	Type 5		Type 1	Type 2	Type 3	Type 4	Type 5
Ω_1	$\mathcal{A}_{35}$	Roy and Roth (2018)	M23K	32.3	15.5	4.9		28.4	ε_1 of $\mathcal{A}_{35}$	27.6	13.3	3.8		
	$\mathcal{A}_{07}$	Wang et al. (2017)	M23K	65.1	73.9	55.8		66.2	ε_2 of $\mathcal{A}_{07}$	54.8	64.0	47.7		
	$\mathcal{A}_{21}$	Lin et al. (2021)	M23K	74.7	76.7	26.6		74.1	ε_3 of $\mathcal{A}_{21}$	63.9	65.9	20.8		
Ω_2	$\mathcal{A}_{20}$	Cao et al. (2021)	DRAW-1K	36.4	36.4	28.9		44.4*	ε_4 of $\mathcal{A}_{20}$	34.5	34.5	26.7		
Ω_3	$\mathcal{A}_{27}$	Yu et al. (2023)	M23K	74.6	89.8	65.1		77.7	t_{40} of $\mathcal{A}_{25}$	94.5	81.6	97.3		
	$\mathcal{A}_{29}$	Meng et al. (2023)	DIR-AWPs			84.5*		84.5*	t_{41} of $\mathcal{A}_{29}$			84.5*		
	$\mathcal{A}_{30}$	Huang et al. (2023)	GCP-PS					81.5*	t_{42} of $\mathcal{A}_{30}$					81.5*
Ω_4	$\mathcal{A}_{25}$	Yu et al. (2022)	TnD1K	83.0	26.2		88.1	80.8	t_{38} of $\mathcal{A}_{25}$	87.3	32.1		90.5	

Ω_s represents "Approach" and $\mathcal{A}_s$ represents "Algorithm" in order to save the space of the table. "Type 6" does not appear in the table because no algorithm solve the problems in this type yet. The numbers with superscript * are from the reference paper, while the other data are produced by this paper.

results obtained through ablation. In the context of solving algorithms, each ablation test can be viewed as evaluating an algorithm with a new transform, as removing a method or feature alters the original structure. Consequently, the results of ablation experiments are inherently encompassed within PCC outcomes. Second, the PCC generates additional insights that ablation experiments cannot. Specifically, it analyzes the effectiveness of individual transforms on different problem types, offering a level of analysis that ablation methods are not designed to provide. Ablation experiments are generally limited to deep learning algorithms, which cannot always be decomposed into distinct components, thus restricting their scope.

The PCC is explicitly designed for the systematic and detailed evaluation of approaches, algorithms, and transforms, establishing a new evaluation paradigm. It allows for a deeper and more nuanced analysis compared to traditional methods. However, it is important to note that the PCC is only applicable to algorithms that can be broken down into component parts.

7.2. Future research directions

The theoretical review on solving algebra problems reveals six research directions.

First, analyzing the solving capacities of approaches is a valuable theory problem. The state graph of an approach decides the theoretical ability that the approach can solve APs; the transforms tell the technique ability of the approach. The prior survey papers (Zhang et al., 2021) mainly emphasized on summarizing the transforms (techniques) of understanding problems, while they cannot reveal the solving capacity of an approach. This paper defined the state structure of each approach. Thus, we can study the theoretical capacity of each approach by analyzing the representation ability of states of constructing the approach. Along this thinking, we can pose a question of what kind of states can represent what types of the algebra problems well.

Second, developing solving algorithms that can meet the requirements of the end users is another urgent research direction. Many algorithms emphasize on the accuracy only. As you know, the accuracy is not all the requirements that end users want from solving algorithms. Hence, it is high time to explore that the need of end users and that end users want which features of the solving algorithms. Thus, it is a valuable research problem how we can achieve these features. We should develop the algorithms with the required properties in order to realize the wide application. One of main objectives of developing solving algorithms is to use the algorithms as the core technology for developing intelligent tutoring systems. For this objective, we should develop the algorithms that can generate humanoid solutions. Unfortunately, many algorithms developed in the recent years did not address this requirement at all. In the future work, researchers are expected to strengthen the efforts on this direction.

Third, another research direction is to widen the application of perspective confusion comparison. Most of algorithms compare with the baseline algorithms on some representative datasets. Such comparisons do not reveal that the algorithm can solve what types of problems in what performance. This paper just gave some samples of perspective confusion comparison, not a comprehensive comparison due to that it is not the goal of this paper. Hence, we plan to accomplish a comprehensive perspective confusion comparison for all the representative transforms, algorithms and approaches on more datasets in the near future.

Fourth, extending the solvable scope is a research direction. Among the selected 47 algorithms, so far only one paper addressed solving the algebra problems with compound object and diagram. Hence, we should emphasize on developing more methods to solve the algebra problems with compound objects and diagrams. Currently two papers proposed methods to extract relations from diagrams. One paper extracted relations from diagrams of function problems and to build functions, being

a kind of compound objects. Another paper extracted relations from geometry diagrams. Hence, we expect that more methods will be proposed to acquire the knowledge from various types of diagrams so that we can solve more types of problems with diagrams.

Fifth, building a solving engine not just an algorithm. This paper has showed that some algorithms have the better performances on some specific types of problems. More importantly, this paper has built the aggregated graph of batch of algorithms. On the other hand, Zhou et al. (2020) proposed a state transition algorithm, which was applied in various optimization problems. The above-mentioned three facts inspire us a new way to develop solving engine in the future. Concretely, we borrow the method in Zhou et al. (2020) to find the optimal portion of the aggregated graph to form a solving engine, which integrates the high performance methods from multiple algorithms.

Sixth, a new direction is applying LLMs into AP-solving. In the recent years, LLMs (large language models), such as GPT-4 (Generative Pretrained Transformer) (OpenAI, 2023) or PaLM-2 (Pretrained Autoregressive Language Model) (Bi et al., 2020) have been applying into building algorithms for solving math word problems (Zhou et al., 2023; Zhu et al., 2023; Zong & Krishnamachari, 2023). Once BERT, the first LLM, was published in 2018, researchers applied it into building solving algorithms and they even built MWP-BERT, which is special for solving math word problems (Liang et al., 2022). They mainly used BERT (or specialized BERT) as tool of converting problems text into vector sequence of problem text to replace the previous word2vec (Liang et al., 2022; Lyu et al., 2023; Yu et al., 2023). With the success of ChatGPT, some researchers proposed to use LLMs as QA (Question and Answer) tool. Following this idea, Zong and Krishnamachari published a paper that tested the performances of GPT-3 answers some questions related to the problems solving (Zong & Krishnamachari, 2023). Zhou et al. (2023) uses LLM as the main base of solving math problems, which is another way of applying LLMs. This paper explores the effect of code on enhancing LLMs' reasoning capability by introducing different constraints on the Code Usage Frequency of GPT-4 Code Interpreter. It found that its success can be largely attributed to its powerful skills in generating and executing code, evaluating the output of code execution, and rectifying its solution when receiving unreasonable outputs. Based on this cognition, they proposed a new prompting method, explicit code-based self-verification (CSV), to further boost the mathematical reasoning potential of GPT-4 Code Interpreter. Pan et al. (2023) conducts a comprehensive evaluation of 11 prominent open-source and proprietary foundation models (LLMs, LLMs augmented with tools, and LMMs), and early experiments with GPT-4V. The evaluation finds that the best performing model, Multimodal Bard, achieves only 58 % of human performance (34.8 % vs 60.3 %), indicating ample room for further improvement. These papers have shown that LLMs can improve the performance of solving math word problems, but LLMs' competency in solving math word problems is not known yet. These papers show us a new research direction, which is how to make use LLMs' competency and the traditional methods together to build the high-performance solving algorithms.

CRedit authorship contribution statement

Xinguo Yu: Conceptualization, Methodology, Writing – review & editing, Supervision, Project administration, Funding acquisition; **Weina Cheng:** Data curation, Writing – original draft, Formal analysis; **Chuanzhi Yang:** Investigation, Data curation, Writing – review & editing, Visualization; **Ting Zhang:** Formal analysis, Writing – review & editing, Validation.

Data availability

The datasets used in this paper are available on our web page.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is partially supported by the general Project of the National Natural Science Foundation of China (Grant No: 62277022).

Appendix A. Descriptions of the 17 identified states and 42 identified transforms

This section presents the state transform graphs (STG) for the 47 representative algorithms.

$\mathcal{S}_{00}$ =: **Input APs**: *Input APs* is the state that algebra problems are input into algorithms, including text and diagrams.

$\mathcal{S}_{01}$ =: **Math Expression**: It first appeared in Hosseini et al. (2014) in 2014, being the verbal math expression to express the answer key in numbers and four operators.

$\mathcal{S}_{02}$ =: **Linear Equation Set**: It first appeared in Kushman et al. (2014) in 2014. This becomes a popular state quickly because the available math tools can solve a set of linear equations.

$\mathcal{S}_{03}$ =: **Expression Tree**: It first appeared in Roy and Roth (2015) in 2015. This state is close to math expression because they can convert each other by simple procedures.

$\mathcal{S}_{04}$ =: **Expression Trees**: It first appeared in Koncel-Kedziorski et al. (2015) in 2015, being a state of multiple math trees, which can be used to solve the algebra problems with multiple unknowns.

$\mathcal{S}_{05}$ =: **Formal Language**: It was proposed in Shi et al. (2015) in 2015. DOL is the formal language used in Shi et al. (2015), being a structured semantic representation of natural language text.

$\mathcal{S}_{06}$ =: **Vector Text**: It first appeared in Wang et al. (2017) in 2017, being the vector sequence of the text. The word2vec (or word embedding) converts text into vector sequence. The most important advantage is that the vector computing technology can be applied into solving algebra problems after problem description can be converted into vector representation.

$\mathcal{S}_{07}$ =: **Vector Single Linear Equation**: It first appeared in Wang et al. (2017) in 2017, which is the output of an RNN-based seq2seq model. It directly transforms vector text into a math equation template, being a single equation containing one unknown variable. More algorithms use various transforms to generate linear equations (Kim et al., 2020; Liu et al., 2019).

$\mathcal{S}_{08}$ =: **Vector Expression Tree**: It is an expression tree generated by a seq2seq neural network, which is equivalent to a math expression, first appeared in Wang et al. (2018a), more papers used expression tree (Li et al., 2020).

$\mathcal{S}_{09}$ =: **Relation Set**: It was first proposed in Yu et al. (2019) in 2019. It is similar to equation set but it is superior to it in two aspects, as Yu et al. (2023, 2019) pointed out, which are: 1) compared with equation set, it is easier to obtain; 2) it is more readable than equation set especially by pupils.

$\mathcal{S}_{10}$ =: **Equation Set**: It is first proposed in Yu et al. (2019) in 2019, being a set of equations, in which equations can be high-order equations.

$\mathcal{S}_{11}$ =: **Vector TextGraph**: It is a sequence of vectors, first appeared in Li et al. (2020), encoded from text and graph derived from text, where graph derived from text aims to represent the structural relations of entities of the problem.

$\mathcal{S}_{12}$ =: **Vector Equation Graph**: It is a graph from which multiple equations can be deciphered. *Direct acyclic graph* is first appeared in Cao et al. (2021) in 2021. Cao et al. (2021) proposed to use seq2DAG to produce DAG from vector text.

$\mathcal{S}_{13}$ =: **Vector Operation Tree**: It is a tree in which the nodes can be formulas with arbitrary number of arguments, proposed in Tsai et al.

(2021). The objective is to incorporate domain knowledge into problem solving and produce readable solutions via using vector operation tree.

$\mathcal{S}_{14}$ =: **xRelation Set**: *xRelation set* was first proposed in Yu et al. (2022); an xRelation set is a set of relations and period relations. xRelation set differs from relation set in two aspects. The first one is that it has period relations. The second one is that these period relations form some cliques of relations and each clique forms one function, being a compound object.

$\mathcal{S}_{15}$ =: **xEquation Set**: It is first proposed in Yu et al. (2022), being a set of equations, period equations, and function. The equations in the set can be high-order equations; period equations are equations with period constraints; function is a compound object that can interact with equations.

$\mathcal{S}_{05}$ =: **Output Solution**: It is the state appeared in all the solving algorithms.

We identified 42 transforms from the 28 algorithms, $\Pi = \{t_{01}, t_{02}, t_{03}, \dots, t_{40}\}$. Table 1 lists all the 42 transforms and their basic facts. We explain these transforms briefly for concise. The items of the basic facts are the original sources of the transforms, borne year, the occurrence number (the number that the transform used in the 47 algorithms). We describe the 42 transforms as follows:

t_{01} =: **Math Expression Generator**: It generates math expression from problem text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$, first appeared in Hosseini et al. (2014) in 2014.

t_{02} =: **Math Expression Evaluator**: It evaluates math expression to produce the answer of the given problem, namely transforming from $\mathcal{S}_{01}$ to $\mathcal{S}_{05}$, first appeared in Hosseini et al. (2014) in 2014.

t_{03} =: **Linear Equations Generator**: It generates a set of linear equations from problem text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{02}$, first appeared in Kushman et al. (2014) in 2014.

t_{04} =: **Solver of Linear Equations**: It solves a set of linear equations to produce the answer of the given problem, being an application of Gaussian eliminator, namely transforming from $\mathcal{S}_{02}$ to $\mathcal{S}_{05}$, first appeared in Kushman et al. (2014) in 2014.

t_{05} =: **Expression Tree Producer**: It is a producer that produces an expression tree, being equivalent to an linear equation, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{03}$, proposed in Roy and Roth (2015) in 2015.

t_{06} =: **Expression Tree Traversal**: It produces a math expression by traversing expression tree and replacing vectors with numbers and operands, namely transforming from $\mathcal{S}_{03}$ to $\mathcal{S}_{01}$, appeared in Roy and Roth (2015) in 2015.

t_{07} =: **Expression Trees Producer**: It produces multiple expression trees, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{04}$, proposed in Koncel-Kedziorski et al. (2015) in 2015. This transform enumerates all the possible equation trees by integer linear programming.

t_{08} =: **Expression Tree Identifier**: It identifies the wanted expression tree by scoring the likelihood of equation trees by learning discriminative models trained from a small number of word problems and their solutions and selecting the tree with the highest score, namely transforming from $\mathcal{S}_{04}$ to $\mathcal{S}_{03}$, proposed in Koncel-Kedziorski et al. (2015) in 2015.

t_{09} =: **QP Equations Generator**: It generates a set of linear equations from problem text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{02}$, proposed in Zhou et al. (2015) in 2015. This transform improves t_{03} by using a QP (quadratic programming) to instantiate the selected template. t_{10} =: **DOL Tree Producer**: It converts problem text into DOL trees, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{05}$ defined in Shi et al. (2015) in 2015. This transform uses a set of grammar rules to build DOL trees from text.

t_{11} =: **DOL2Expression Generator**: It generates math expression from DOL trees, namely transforming from $\mathcal{S}_{05}$ to $\mathcal{S}_{02}$, first appeared in Shi et al. (2015) in 2015.

t_{12} =: **Word2Vec Converter**: It converts text into vector sequence by a Word2Vec procedure, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{06}$, proposed in Wang et al. (2017) in 2017. The vector text becomes popular soon in the various types of machine learning based algorithms.

t_{13} = **RNN Equations Generator**: It generates equations from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{07}$ by RNN (Recurrent Neural Network), proposed in Wang et al. (2017) in 2017. The gated recurrent units (GRU) and long short-memory (LSTM) cells are used for encoding and decoding, respectively.

t_{14} = **Equations Retrieval**: It generates equations from vector text through similarity matching, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{01}$, proposed in Wang et al. (2017) in 2017.

t_{15} = **V2T Expression Converter**: It converts vector expression to math expression, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{01}$, used in Wang et al. (2017) in 2017.

t_{16} = **RL Vector Expression Generator**: It generates vector expression from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{07}$ by RL (Reinforcement Learning), proposed in Huang et al. (2018a) in 2018. This transform improves t_{17} by incorporating copy and alignment mechanism.

t_{17} = **DQN Vector Expression Generator**: It generates vector expression from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$ by DQN (Deep Q-Network), first appeared in Wang et al. (2018b) in 2018.

t_{18} = **Seq2Tree Generator**: It generates vector tree from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$, first appeared in Liu et al. (2019) in 2019.

t_{19} = **V2E Tree Converter**: It converts vector tree to expression tree, namely transforming from $\mathcal{S}_{08}$ to $\mathcal{S}_{01}$, first appeared in Liu et al. (2019) in 2019.

t_{20} = **tRNN Seq2Tree Generator**: It generates vector expression tree from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$ by tRNN (template-based Recursive Neural Network), proposed in Wang et al. (2019) in 2019.

t_{21} = **Goal-Driven Seq2Tree Producer**: It produces vector tree from vector text by using goal-driven tree-structured neural model, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$, proposed in Xie and Sun (2019) in 2019.

t_{22} = **S^2 Relation Extractor**: It acquires explicit relations from annotated problem text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{09}$, proposed in Yu et al. (2019) in 2019. S^2 extractor uses a pool of S^2 models (S^2 stands for syntax-semantics) to extract explicit relations one by one from the annotated text of describing a problem.

t_{23} = **Equation Set Producer**: It produces an equation set, namely transforming from $\mathcal{S}_{09}$ to $\mathcal{S}_{10}$, first appeared in Yu et al. (2019) in 2019, converting a relation set into an equation set by replacing entities with defined variables.

t_{24} = **Equation Set Solver**: It solves equation set to produce the answer of the given problem, namely transforming from $\mathcal{S}_{10}$ to $\mathcal{S}_{05}$, first appeared in Yu et al. (2019) in 2019.

t_{25} = **EPT Vector L-Equation Generator**: It generates vector linear equation from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{07}$ by using EPT (Expression-Pointer Transformer), first appeared in Kim et al. (2020) in 2020. This transform uses “expression” token and operand-context pointers when generating vector linear equation and adopts a pretrained language model as the encoder.

t_{26} = **TextGraph Encoder**: It first acquires the graph from text and then encodes text and graph into a vector sequence, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{11}$, proposed in Li et al. (2020).

t_{27} = **Graph2tree Decoder**: It generates vector tree from the vector sequence of TextGraph, namely transforming from $\mathcal{S}_{11}$ to $\mathcal{S}_{08}$, such as equation tree or expression tree in AP-solving tasks, proposed in Li et al. (2020).

t_{28} = **Seq2DAG Generator**: It generates direct acyclic graph from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{12}$, first appeared in Cao et al. (2021) in 2021.

t_{29} = **DAG Traversal**: It produces a set of equations by traversing direct acyclic graph where equations may be high-order in appearance but they are linear essentially, namely transforming from $\mathcal{S}_{12}$ to $\mathcal{S}_{02}$, first appeared in Cao et al. (2021) in 2021.

t_{30} = **HMS Operation Tree Generator**: It generates seq2seq operation tree from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{13}$ by HMS (Hierarchical Math Solver), first appeared in Lin et al. (2021) in 2021.

t_{31} = **VoS² Relation Extractor**: It uses VoS² method to extract relations from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{09}$ by using vector syntax semantics method, first appeared in Lyu and Yu (2021) in 2021 and further developed in Lyu et al. (2023) in 2023. VoS² extractor is the improved version of S^2 extractor.

t_{32} = **Vector Operation Tree Producer**: It is a seq2seq neural network that produces operation tree in Tsai et al. (2021), namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{13}$.

t_{33} = **Operation Tree Evaluator**: It is a procedure that converts an operation tree into an expression tree, namely transforming from $\mathcal{S}_{13}$ to $\mathcal{S}_{03}$. The algorithm in Tsai et al. (2021) should have this procedure, but it did not present it.

t_{34} = **Tree-RNN Generator**: It is a neural network to produce an equation from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{02}$, first appeared in Zaporozhets et al. (2021) in 2021. This transform scores the generated candidate equations using Tree-RNN (Tree-structured Recurrent Neural Network).

t_{35} = **S²P Relation Extractor**: It extracts universal and period relations from text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{14}$, first appeared in Yu et al. (2022) in 2022.

t_{36} = **L² Relation Extractor**: It acquires period relation from problem diagrams, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{14}$, first appeared in Yu et al. (2022) in 2022, which is designed to acquire the group of period relations from diagrams. This is the first transform that can acquire relations from diagrams for algebra problems.

t_{37} = **xEquation Set Producer**: It is a producer that forms an xequation set, namely transforming from $\mathcal{S}_{14}$ to $\mathcal{S}_{15}$, first appeared in Yu et al. (2022) in 2022. This transform does two main tasks. The first one is to identify the appearances of each entity and assign variables to entities. The second one is to state the cliques of period relations and build a function for each clique.

t_{38} = **xEquation Set Solver**: It solves xequation set to produce the answer of the given problem, namely transforming from $\mathcal{S}_{15}$ to $\mathcal{S}_{05}$, first appeared in Yu et al. (2022) in 2022.

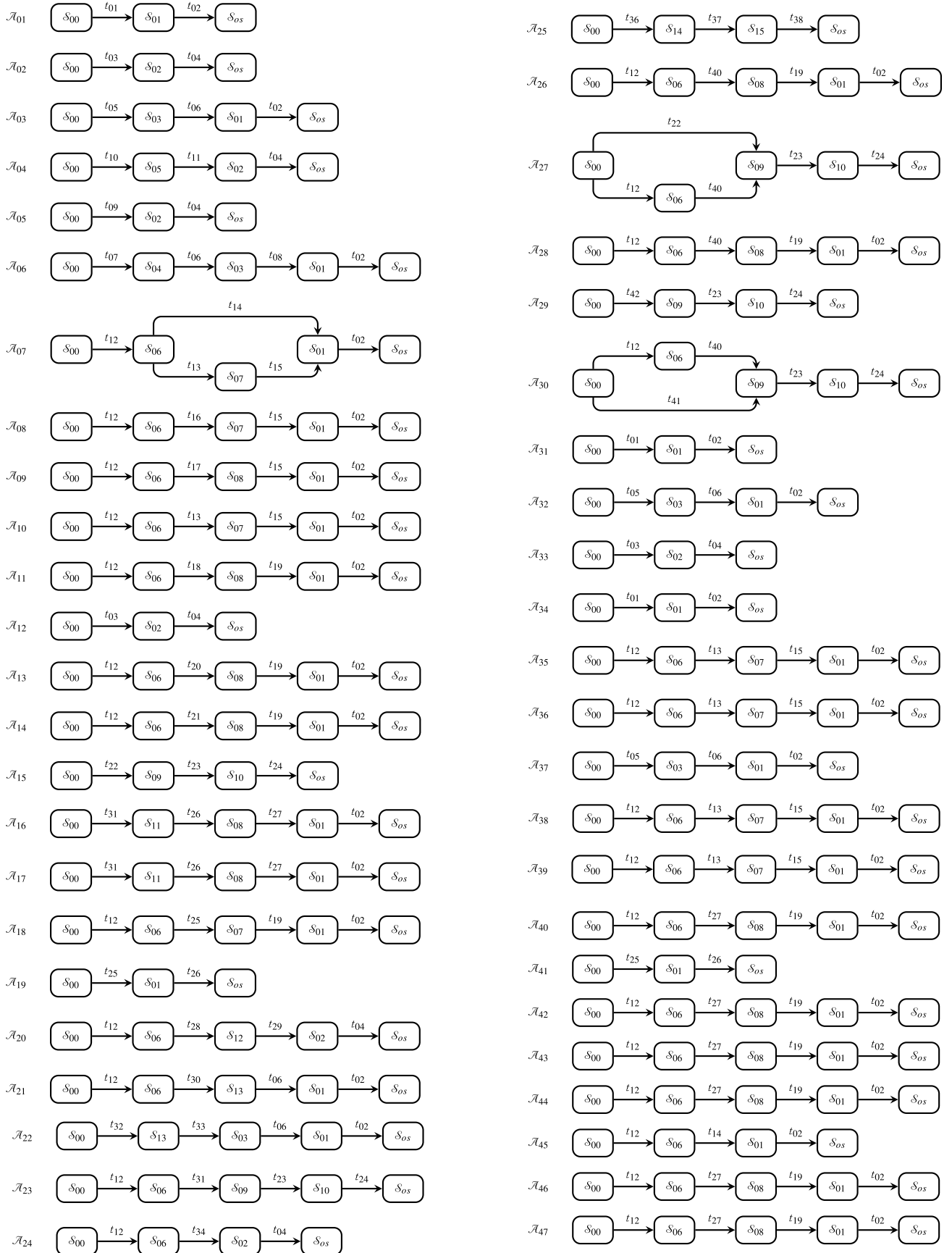
t_{39} = **HGEN Vector Expression Generator**: It is a neural network to generate vector expression from vector text by HGEN (hierarchical heterogeneous graph encoding), namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{08}$, appeared in Zhang et al. (2022) in 2022.

t_{40} = **Neural Network Miner**: It is a miner that mines implicit relation from vector text, namely transforming from $\mathcal{S}_{06}$ to $\mathcal{S}_{09}$, proposed in Yu et al. (2023) in 2023.

t_{41} = **DI Relation Extractor**: It extracts DI (Deep Implicit) relations from problem text, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{09}$, proposed in Meng et al. (2023) in 2023. This transform is designed to acquire the deep implicit relations by using qualia syntax-semantic model, extending the scope of problems that solving algorithms can solve.

t_{42} = **GD Relation Extractor**: It extracts relations from geometry diagram, the partial description of the given problem, namely transforming from $\mathcal{S}_{00}$ to $\mathcal{S}_{09}$, proposed in Huang et al. (2023) in 2023. This transform is designed to acquire the relations from geometry diagrams by using syntax-semantics diagram understanding, extending the scope of problems that solving algorithms can solve.

Appendix B. State transform graphs for All 47 algorithms



References

- Bi, B., Li, C., Wu, C., Yan, M., Wang, W., Huang, S., Huang, F., & Si, L. (2020). PALM: Pre-training an autoencoding & autoregressive language model for context-conditioned generation. In *EMNLP 2020*. <https://doi.org/10.48550/arXiv.2004.07159>.
- Campbell, D. J. (1995). Testing heuristics: We have it all wrong. *Journal of Heuristics*, 1(1), 33–42.
- Cao, Y., Hong, F., Li, H., & Luo, P. (2021). A bottom-up DAG structure extraction model for math word problems. In *Thirty-fifth AAAI conference on artificial intelligence* (pp. 39–46). (vol. 35). <https://doi.org/10.1609/aaai.v35i1.16075>.
- Carifio, J., & Nasser, R. (1994). Algebra word problems: A review of the theoretical models and related research literature. In *The annual conference of the american educational research association*.
- Faldu, K., Sheth, A., Kikani, P., Gaur, M., & Avasthi, A. (2021). Towards tractable mathematical reasoning: Challenges, strategies, and opportunities for solving math word problems. [arXiv:2111.05364](https://arxiv.org/abs/2111.05364).
- Gan, W., Yu, X., & Wang, M. (2019a). Automatic understanding and formalization of plane geometry proving problems in natural language: A supervised approach. *International Journal on Artificial Intelligence Tools*, 28(4), 1940003. <https://doi.org/10.1142/S0218213019400037>.
- Gan, W., Yu, X., Zhang, T., & Wang, M. (2019b). Automatically proving plane geometry theorems stated by text and diagram. *International Journal of Pattern Recognition and Artificial Intelligence*, 33(7), 1940003.1–1940003.26. <https://doi.org/10.1142/S0218001419400032>.
- Gill, A. (1960). Comparison of finite-state models. *IRE Transactions on Circuit Theory*, 7(2), 178–179.
- He, B., Yu, X., Jian, P., & Zhang, T. (2020). A relation based algorithm for solving direct current circuit problems. *Applied Intelligence*, (11). <https://doi.org/10.1007/s10489-020-01667-7>.
- Hosseini, M. J., Hajishirzi, H., Etzioni, O., & Kushman, N. (2014). Learning to solve arithmetic word problems with verb categorization. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)* (pp. 523–533). Doha, Qatar: Association for Computational Linguistics. <https://doi.org/10.3115/v1/P14-1026>.
- Huang, D., Liu, J., Lin, C.-Y., & Yin, J. (2018a). Neural math word problem solver with reinforcement learning. In *Proceedings of the 27th international conference on computational linguistics, Santa Fe, New Mexico* (pp. 213–223).
- Huang, D., Shi, S., Yin, J., & Lin, C.-Y. (2017). Learning fine-grained expressions to solve math word problems. In *Proceedings of the 2017 conference on empirical methods in natural language processing* (pp. 805–814). Copenhagen, Denmark: Association for Computational Linguistics. <https://doi.org/10.18653/v1/D17-1084>.
- Huang, D., Yao, J.-G., Lin, C.-Y., Zhou, Q., & Yin, J. (2018b). Using intermediate representations to solve math word problems. In *Proceedings of the 56th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 419–428). <https://doi.org/10.18653/v1/P18-1039>.
- Huang, L., Yu, X., Niu, L., & Feng, Z. (2023). Solving algebraic problems with geometry diagrams using syntax-semantics diagram understanding. *Computers, Materials and Continua*, 77(1), 517–539. <https://doi.org/10.32604/cmc.2023.041206>.
- Jian, P., Sun, C., Yu, X., He, B., & Xia, M. (2019). An end-to-end algorithm for solving circuit problems. *International Journal of Pattern Recognition and Artificial Intelligence*, 33, 1940004.1–1940004.21. <https://doi.org/10.1142/S0218001419400044>.
- Jie, Z., Li, J., & Lu, W. (2022). Learning to reason deductively: Math word problem solving as complex relation extraction. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 5944–5955). <https://doi.org/10.18653/v1/2022.acl-long.410>.
- Kim, B., Ki, K. S., Lee, D., & Gwon, G. (2020). Point to the Expression: Solving Algebraic Word Problems using the Expression-Pointer Transformer Model. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* (pp. 3768–3779). <https://doi.org/10.18653/v1/2020.emnlp-main.308>.
- Koncel-Kedziorski, R., Hajishirzi, H., Sabharwal, A., Etzioni, O., & Ang, S. D. (2015). Parsing algebraic word problems into equations. *Transactions of the Association for Computational Linguistics*, 3(1), 585–597. https://doi.org/10.1162/tacl_a.00160.
- Kotwal, H., & Patnaik, G. K. (2019). A review of automatic math word problem solving techniques. *International Journal of Management, Technology And Engineering*, 9(1), 1544–1548.
- Kushman, N., Artzi, Y., Zettlemoyer, L., & Barzilay, R. (2014). Learning to automatically solve algebra word problems. In *Proceedings of the 52nd annual meeting of the association for computational linguistics* (pp. 271–281). Baltimore, Maryland, USA: Association for Computational Linguistics. <https://doi.org/10.3115/v1/P14-1026>.
- Li, J., Wang, L., Zhang, J., Wang, Y., Dai, B. T., & Zhang, D. (2019). Modeling intra-relation in math word problems with different functional multi-head attentions. In *Proceedings of the 57th annual meeting of the association for computational linguistics* (pp. 6162–6167). <https://doi.org/10.18653/v1/P19-1619>.
- Li, S., Wu, L., Feng, S., Xu, F., Xu, F., & Zhong, S. (2020). Graph-to-tree neural networks for learning structured input-output translation with applications to semantic parsing and math word problem. In *Findings of the association for computational linguistics: EMNLP 2020* (pp. 2841–2852). <https://doi.org/10.18653/v1/2020.findings-emnlp.255>.
- Li, Z., Zhang, W., Yan, C., Zhou, Q., Li, C., Liu, H., & Cao, Y. (2022). Seeking patterns, not just memorizing procedures: Contrastive learning for solving math word problems. In *Findings of the association for computational linguistics: ACL 2022* (pp. 2486–2496). <https://doi.org/10.18653/v1/2022.findings-acl.195>.
- Liang, Z., Zhang, J., Wang, L., Qin, W., Lan, Y., Shao, J., & Zhang, X. (2022). MWP-BERT: Numeracy-augmented pre-training for math word problem solving. In *Findings of the association for computational linguistics: NAACL 2022* (pp. 997–1009). <https://doi.org/10.18653/v1/2022.findings-naacl.74>.
- Liang, Z., Zhang, J., Wang, L., Wang, Y., Shao, J., & Zhang, X. (2023). Generalizing math word problem solvers via solution diversification. In *Proceedings of the thirty-seventh AAAI conference on artificial intelligence and thirty-fifth conference on innovative applications of artificial intelligence and thirteenth symposium on educational advances in artificial intelligence*. <https://doi.org/10.1609/aaai.v37i1.1.26548>.
- Lin, X., Huang, Z., Zhao, H., Chen, E., Liu, Q., Lian, D., Li, X., & Wang, H. (2023). Learning relation-enhanced hierarchical solver for math word problems. *IEEE Transactions on Neural Networks and Learning Systems*, (pp. 1–15). <https://doi.org/10.1109/TNNLS.2023.3272114>.
- Lin, X., Huang, Z., Zhao, H., Chen, E., Liu, Q., Wang, H., & Wang, S. (2021). HMS: A hierarchical solver with dependency-enhanced understanding for math word problem. In *The thirty-fifth AAAI conference on artificial intelligence (AAAI-21)* (pp. 4232–4240). AAAI. <https://doi.org/10.48550/arXiv.2105.04165>.
- Ling, W., Yogatama, D., Dyer, C., & Blunsom, P. (2017). Program induction by rationale generation: Learning to solve and explain algebraic word problems. In *Proceedings of the 55th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 158–167). <https://doi.org/10.18653/v1/P17-1015>.
- Liu, Q., Guan, W., Li, S., & Kawahara, D. (2019). Tree-structured decoding for solving math word problems. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)* (pp. 2370–2379). <https://doi.org/10.18653/v1/D19-1241>.
- Lu, P., Qiu, L., Yu, W., Welleck, S., & Chang, K.-W. (2023). A survey of deep learning for mathematical reasoning. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (p. 14605–14631). <https://doi.org/10.18653/v1/2023.acl-long.817>.
- Luan, S., & Li, W. (2001). Formal specification for evolution of algorithm and its properties. *Journal of Software*, 12(4), 493–498.
- Lyu, X., & Yu, X. (2021). Solving explicit arithmetic word problems via using vectorized syntax-semantics model. In *2021 IEEE international conference on teaching, assessment, and learning for engineering (TALE)* (pp. 01–07). <https://doi.org/10.1109/TALE52509.2021.9678714>.
- Lyu, X., Yu, X., & Peng, R. (2023). Vector relation acquisition and scene knowledge for solving arithmetic word problems. *Journal of King Saud University - Computer and Information Sciences*, 35(8), 101673. <https://doi.org/10.1016/j.jksuci.2023.101673>.
- Mandal, S. (2019). Solving arithmetic word problems by object oriented modeling and query-based information processing. *International Journal on Artificial Intelligence Tools*, 28(4), 1940002. (23 pages). <https://doi.org/10.1142/S0218213019400025>.
- Mandal, S., & Naskar, S. K. (2019). Solving arithmetic mathematical word problems: A review and recent advancements. In *Information Technology and Applied Mathematics (vol.699, pp. 95–114)*. Singapore: Springer Singapore.
- Mandal, S., & Naskar, S. K. (2021). Classifying and solving arithmetic math word problems - an intelligent math solver. *IEEE Transactions on Learning Technologies*, 14(1), 28–41. <https://doi.org/10.1109/TLT.2021.3057805>.
- Mandal, S., Sekh, A. A., & Naskar, S. K. (2020). Solving arithmetic word problems: A deep learning based approach. *Journal of Intelligent & Fuzzy Systems*, 39(2), 2521–2531.
- Mendling, J., Depaire, B., & Leopold, H. (2021). Theory and practice of algorithm engineering. [arXiv:2107.10675](https://arxiv.org/abs/2107.10675).
- Meng, H., Yu, X., He, B., & Huang, L. (2023). Solving arithmetic word problems of entailing deep implicit relations by qualia syntax-semantic model. *Computers Materials & Continua*, 77(1), 541–555. <https://doi.org/10.32604/cmc.2023.041508>.
- Mitra, A., & Baral, C. (2016). Learning to use formulas to solve simple arithmetic problems. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 2144–2153). <https://doi.org/10.18653/v1/P16-1202>.
- Mukherjee, A., & Garain, U. (2008). A review of methods for automatic understanding of natural language mathematical problems. *Artificial Intelligence Review*, 29(2), 93–122. <https://doi.org/10.1007/s10462-009-9110-0>.
- OpenAI (2023). GPT-4 technical report.
- Pan, L., Hritik, B., Tony, X., Jiacheng, L., Chunyuan, L., Hannaneh, H., Hao, C., Kai-Wei, C., Michel, G., & Jianfeng, G. (2023). MathVista: Evaluating math reasoning in visual contexts with GPT-4V, Bard, and other large multimodal models.
- Qin, J., Liang, X., Hong, Y., Tang, J., & Lin, L. (2021). Neural-symbolic solver for math word problems with auxiliary tasks. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing* (pp. 5870–5881). <https://doi.org/10.18653/v1/2021.acl-long.456>.
- Rehman, T., Khan, S., Hwang, G.-J., & Abbas, M. A. (2019). Automatically solving two-variable linear algebraic word problems using text mining. *Expert Systems*, 36(2), 1–11. <https://doi.org/10.1111/exsy.12358>.
- Roy, S., & Roth, D. (2015). Solving general arithmetic word problems. In *Conference: Proceedings of the 2015 conference on empirical methods in natural language processing* (pp. 1743–1752). <https://doi.org/10.18653/v1/D15-1202>.
- Roy, S., & Roth, D. (2018). Mapping to declarative knowledge for word problem solving. *Transactions of the Association of Computational Linguistics*, 6, 159–172.
- Roy, S., Upadhyay, S., & Roth, D. (2016). Equation parsing: Mapping sentences to grounded equations. In *Proceedings of the 2016 conference on empirical methods in natural language processing* (p. 1088–1097). <https://doi.org/10.18653/v1/d16-1117>.
- Shi, S., Wang, Y., Lin, C.-Y., Liu, X., & Rui, Y. (2015). Automatically solving number word problems by semantic parsing and reasoning. In *Proceedings of the 2015 conference on empirical methods in natural language processing* (pp. 1132–1142). Lisbon, Portugal: Association for Computational Linguistics. <https://doi.org/10.18653/v1/D15-1135>.
- Tsai, S.-h., Liang, C.-C., Wang, H.-M., & Su, K.-Y. (2021). Sequence to general tree: Knowledge-guided geometry word problem solving. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 2: Short papers)* (pp. 964–972). <https://doi.org/10.18653/v1/2021.acl-short.121>.
- Upadhyay, S., Chang, M.-W., Chang, K.-W., & Yih, W.-t. (2016). Learning from explicit and implicit supervision jointly for algebra word problems. In *Proceedings of the 2016*

- conference on empirical methods in natural language processing (pp. 297–306). <https://doi.org/10.18653/v1/D16-1029>.
- Wang, L., Wang, Y., Cai, D., Zhang, D., & Liu, X. (2018a). Translating a math word problem to an expression tree. In *Proceedings of the 2018 conference on empirical methods in natural language processing (EMNLP)* (pp. 1064–1069). <https://doi.org/10.18653/v1/D18-1132>.
- Wang, L., Zhang, D., Gao, L., Song, J., Guo, L., & Shen, H. T. (2018b). MathDQN: Solving arithmetic word problems via deep reinforcement learning. In *The thirty-fifth AAAI conference on artificial intelligence (AAAI-18)* (pp. 5545–5552). <https://doi.org/10.1609/aaai.v32i1.11981>.
- Wang, L., Zhang, D., Zhang, J., Xu, X., Gao, L., Dai, B. T., & Shen, H. T. (2019). Template-based math word problem solvers with recursive neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 7144–7151). (vol. 33). <https://doi.org/10.1609/aaai.v33i01.33017144>.
- Wang, Y., Liu, X., & Shi, S. (2017). Deep neural solver for math word problems. In *Proceedings of the 2017 conference on empirical methods in natural language processing* (pp. 845–854). <https://doi.org/10.18653/v1/D17-1088>.
- Wu, Q., Zhang, Q., Fu, J., & Huang, X. (2020). A knowledge-aware sequence-to-tree network for math word problem solving. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)* (pp. 7137–7146). <https://doi.org/10.18653/v1/2020.emnlp-main.579>.
- Wu, Q., Zhang, Q., & Wei, Z. (2021a). An edge-enhanced hierarchical graph-to-tree network for math word problem solving. In *Findings of the association for computational linguistics: EMNLP 2021* (pp. 1473–1482). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.findings-emnlp.127>.
- Wu, Q., Zhang, Q., Wei, Z., & Huang, X. (2021b). Math word problem solving with explicit numerical values. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: Long papers)* (pp. 5859–5869). Association for Computational Linguistics.
- Xia, J., & Yu, X. (2021). On the approaches of solving arithmetic problems. In *2021 IEEE international conference on engineering, technology & education (TALE)* (pp. 65–70). <https://doi.org/10.1109/TALE52509.2021.9678859>.
- Xie, Z., & Sun, S. (2019). A goal-driven tree-structured neural model for math word problems. In *Proceedings of the twenty-eighth international joint conference on artificial intelligence (IJCAI-19)* (pp. 5299–5305). <https://doi.org/10.24963/ijcai.2019/736>.
- Yu, X., Lyu, X., Peng, R., & Shen, J. (2023). Solving arithmetic word problems by synergizing syntax-semantics extractor for explicit relations and neural network miner for implicit relations. *Complex & Intelligent Systems*, 9(1), 697–717. <https://doi.org/10.1007/s40747-022-00828-0>.
- Yu, X., Sun, H., & Sun, C. (2022). A relation-centric algorithm for solving text-diagram function problems. *Journal of King Saud University - Computer and Information Sciences*, 34(10, Part B), 8972–8984. <https://doi.org/10.1016/j.jksuci.2022.08.023>.
- Yu, X., Wang, M., Gan, W., He, B., & Ye, N. (2019). A framework for solving explicit arithmetic word problems and proving plane geometry theorems. *International Journal of Pattern Recognition and Artificial Intelligence*, 33(7), 1940005.1–1940005.21.
- Zaporojets, K., Bekoulis, G., Deleu, J., Demeester, T., & Devellder, C. (2021). Solving arithmetic word problems by scoring equations with recursive neural networks. *Expert Systems with Applications*, 174(2), 114704. <https://doi.org/10.1016/j.eswa.2021.114704>.
- Zhang, D., Wang, L., Zhang, L., Dai, B. T., & Shen, H. T. (2020a). The gap of semantic parsing: A survey on automatic math word problem solvers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(9), 2287–2305. <https://doi.org/10.1109/TPAMI.2019.2914054>.
- Zhang, J., Lee, R. K.-W., Lim, E.-P., Qin, W., Wang, L., Shao, J., & Sun, Q. (2021). Teacher-student networks with multiple decoders for solving math word problem. In *Proceedings of the twenty-ninth international joint conference on artificial intelligence*. <https://doi.org/10.24963/ijcai.2020/555>.
- Zhang, J., Wang, L., Lee, R. K.-W., Bin, Y., Wang, Y., Shao, J., & Lim, E.-P. (2020b). Graph-to-tree learning for solving math word problems. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 3928–3937). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.acl-main.362>.
- Zhang, Y., Zhou, G., Xie, Z., & Huang, J. X. (2022). HGEN: Learning hierarchical heterogeneous graph encoding for math word problem solving. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30, 816–828. <https://doi.org/10.1109/TASLP.2022.3145314>.
- Zhou, A., Wang, K., Lu, Z., Shi, W., Luo, S., Qin, Z., Lu, S., Jia, A., Song, L., Zhan, M., & Li, H. (2023). Solving challenging math word problems using GPT-4 code interpreter with code-based self-verification.
- Zhou, L., Dai, S., & Chen, L. (2015). Learn to solve algebra word problems using quadratic programming. In *Proceedings of the 2015 conference on empirical methods in natural language processing* (pp. 817–822). Association for Computational Linguistics. <https://doi.org/10.18653/v1/D15-1096>.
- Zhou, X., Yang, C., & Weihua, G. (2020). The principle of state transition algorithm and its applications. *Acta Automatic Sinica*, 46(11), 2260–2274. <https://doi.org/10.16383/j.aas.c190624>.
- Zhu, X., Wang, J., Zhang, L., Zhang, Y., Huang, Y., Gan, R., Zhang, J., & Yang, Y. (2023). Solving math word problems via cooperative reasoning induced language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)* (pp. 4471–4485). <https://doi.org/10.18653/v1/2023.acl-long.245>.
- Zong, M., & Krishnamachari, B. (2023). Solving math word problems concerning systems of equations with GPT models. *Machine Learning with Applications*, 14, 100506. <https://doi.org/10.1016/j.mlwa.2023.100506>.